

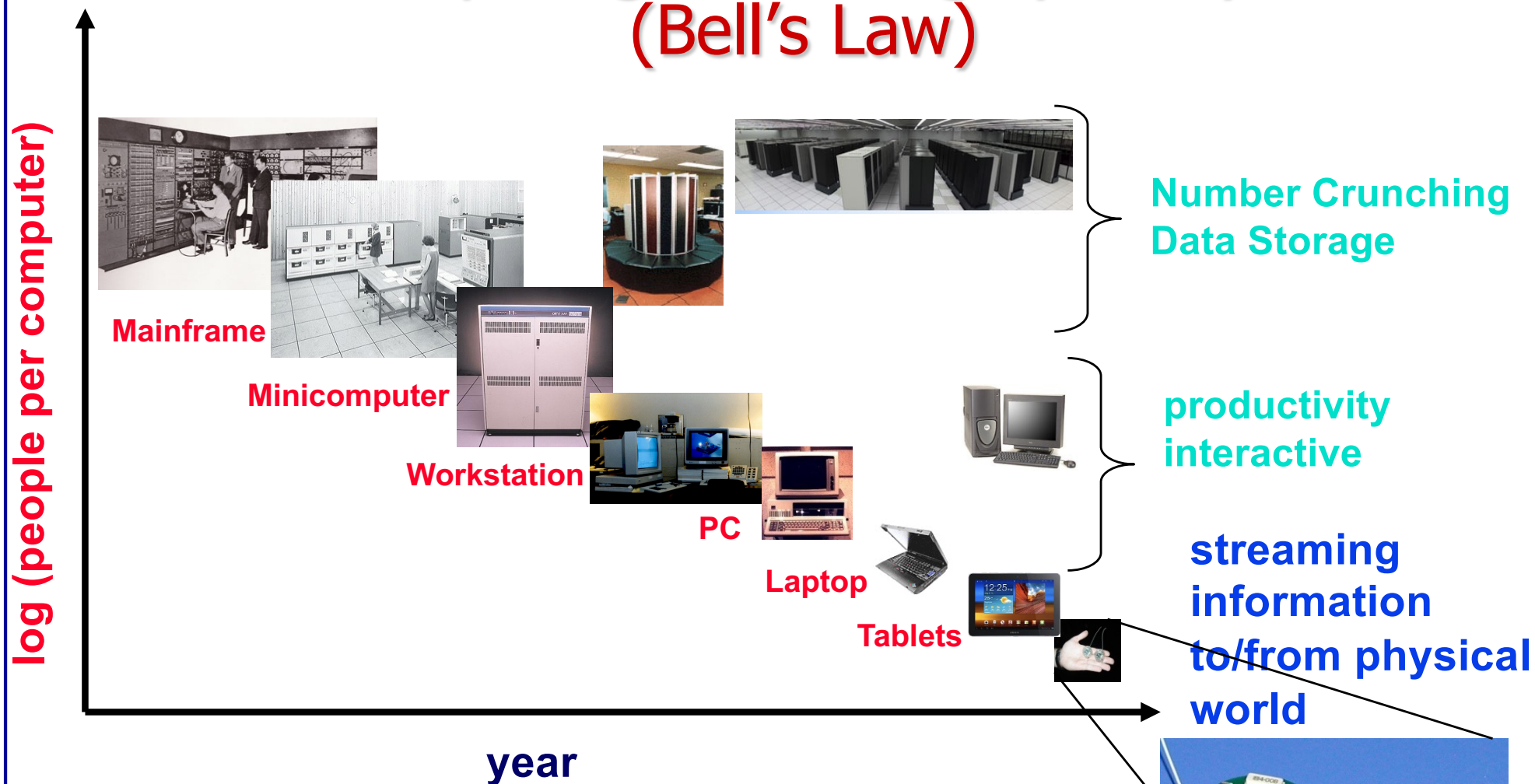
Advanced Computer Systems (CIS 570)

The What! the Why! And a little How!

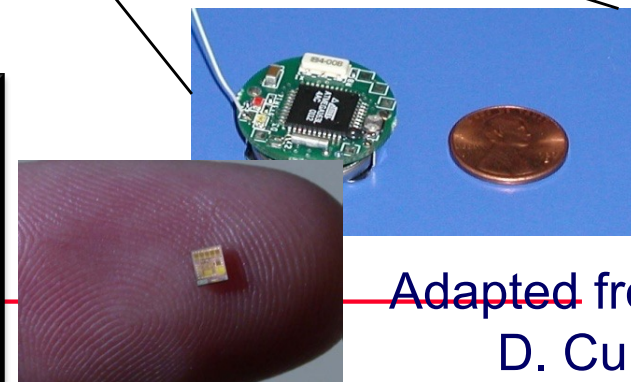
Slides are adopted from:

- Patterson and Hennessy, Computer Architecture a Quantitative Approach (Fifth edition)
- Notes by Dr. O. Mutlu, CMU
- Patterson and Hennessy, Computer Organization and Design: The Hardware/Software Interface, Fourth Edition
- Notes by Dr. M. Geiger, UMA.
- Vincent Heuring and Harry Jordan, Computer Systems design and Architecture, Second Edition

A new computing class roughly every decade (Bell's Law)



"Roughly every decade a new, lower priced computer class forms based on a new programming platform, network, and interface resulting in new usage and the establishment of a new industry."

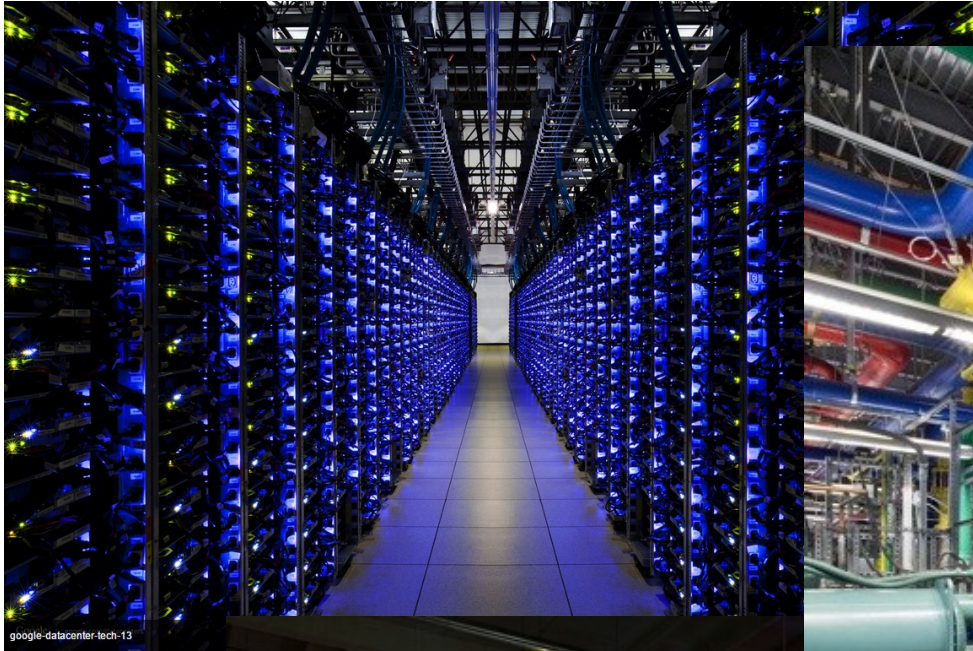


Adapted from
D. Culler²

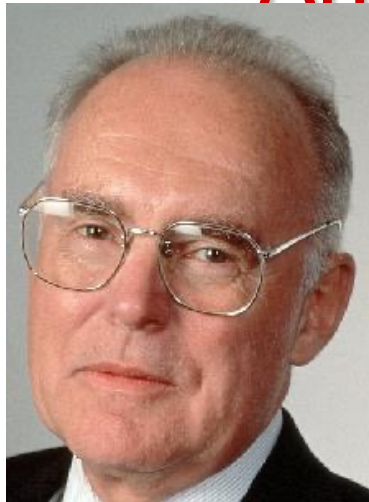
Mobile Devices (Now)



My Other Computer is a Data Center



An Enabler: Moore's Law



Gordon Moore
Intel Cofounder

MOORE'S LAW "Transistor density on integrated circuits doubles about every two years." *

1950s

Silicon
Transistor



1
Transistor

1960s

TTL
Quad Gate



16
Transistors

1970s

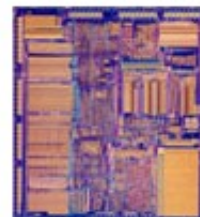
8-bit
Microprocessor



4500
Transistors

1980s

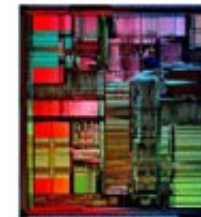
32-bit
Microprocessor



275,000
Transistors

1990s

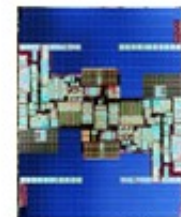
32-bit
Microprocessor



3,100,000
Transistors

2000s

64-bit
Microprocessor



592,000,000
Transistors

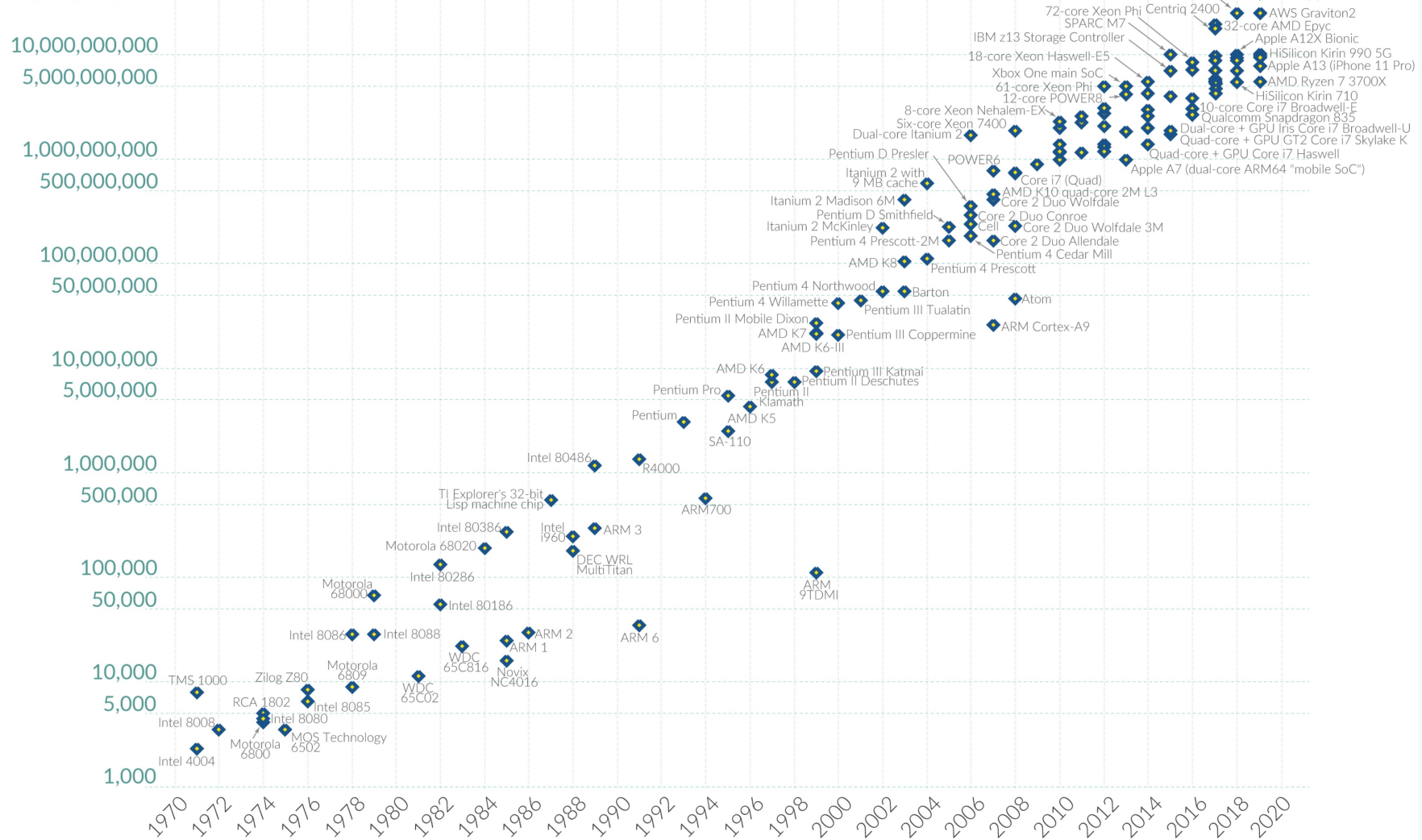
Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years.

This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Transistor count

50,000,000,000

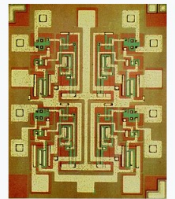


Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under [CC-BY](#) by the authors Hannah Ritchie and Max Roser.

Semiconductor device fabrication



MOSFET scaling (process nodes)

20 μm – 1968

10 μm – 1971

6 μm – 1974

3 μm – 1977

1.5 μm – 1981

1 μm – 1984

800 nm – 1987

600 nm = 1990

600 nm	= 1990
350 nm	1003

350 nm = 1993

250 nm – 1996

180 nm – 1999

130 nm – 2001

90 nm – 2003

65 nm – 2005

45 nm – 2007

32 nm – 2009

28 nm – 2010

22 nm – 2012

14 nm – 2014

10 nm – 2016

7 nm – 2018

5 nm – 2020

3 nm – 2022

2 nm = 2025

Future

1 nm ~ 2027

Computer Technology - Dramatic Change!

- Memory
 - DRAM capacity: 2x / 2 years (since '96);
64x size improvement in last decade.
- Processor
 - Speed 2x / 1.5 years (since '85);
100X performance in last decade.
- Disk
 - Capacity: 2x / 1 year (since '97)
250X size in last decade.

Computer Technology - Dramatic Change!

- Shift to Multi-core (double number of cores):
 - Processor clock speed: 4000 **Mega**Hertz
(4.0 **Giga**Hertz)
 - Memory capacity: 64000 **Mega**Bytes
(64.0 **Giga**Bytes)
 - Disk capacity: 4000 **Giga**Bytes
(4.0 **Tera**Bytes)
 - New units! **Mega** $\Rightarrow$ **Giga**, **Giga** $\Rightarrow$ **Tera**
Order-of-magnitude difference!
-

From ILP to TLP & DLP

- (Almost) All microprocessor companies moving to multiprocessor systems
- Single processors gain performance by exploiting instruction level parallelism (ILP)
- Multiprocessors exploit either:
 - Thread level parallelism (TLP), or
 - Data level parallelism (DLP)

From ILP to TLP & DLP (cont.)

- We've got **tons of infrastructure** for single-processor systems
 - **Algorithms**, languages, compilers, operating systems, architectures, etc.
 - These **don't exactly scale well**
- **Multiprocessor** design: **not as simple** as creating a chip with **1000 CPUs**
 - Task **scheduling/division**
 - **Communication**
 - **Memory** issues
 - Even **programming** → moving from 1 to 2 CPUs is extremely difficult

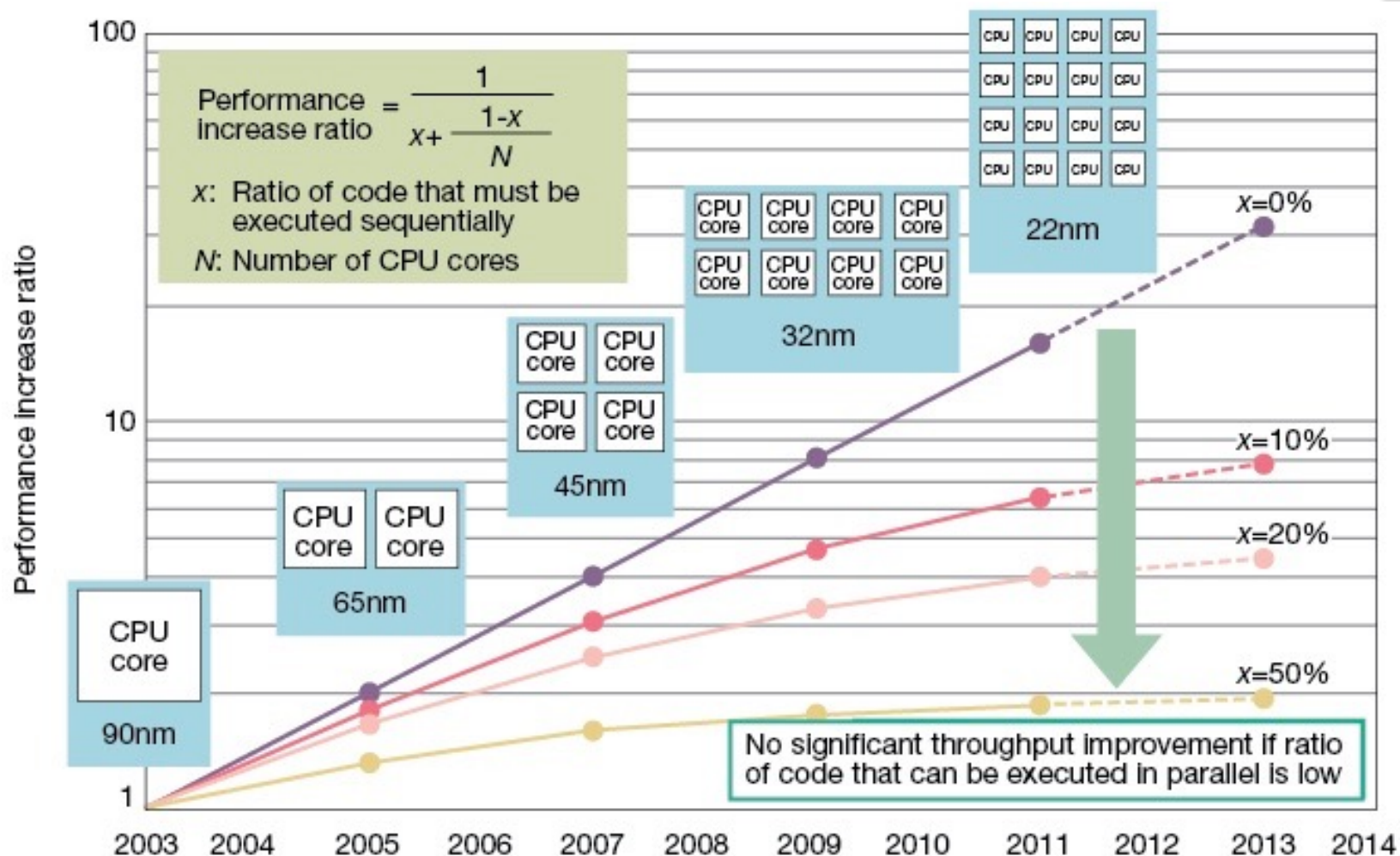
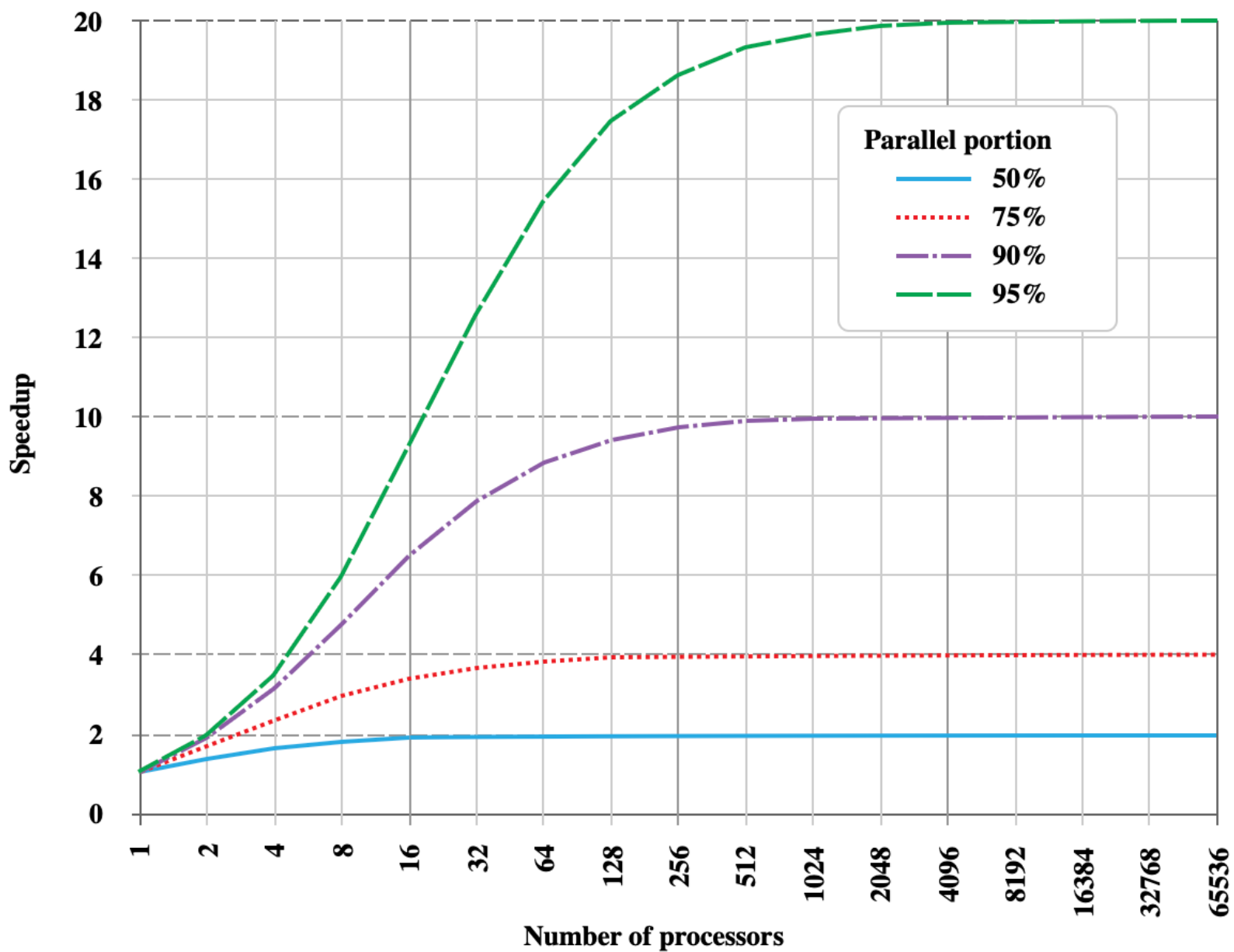
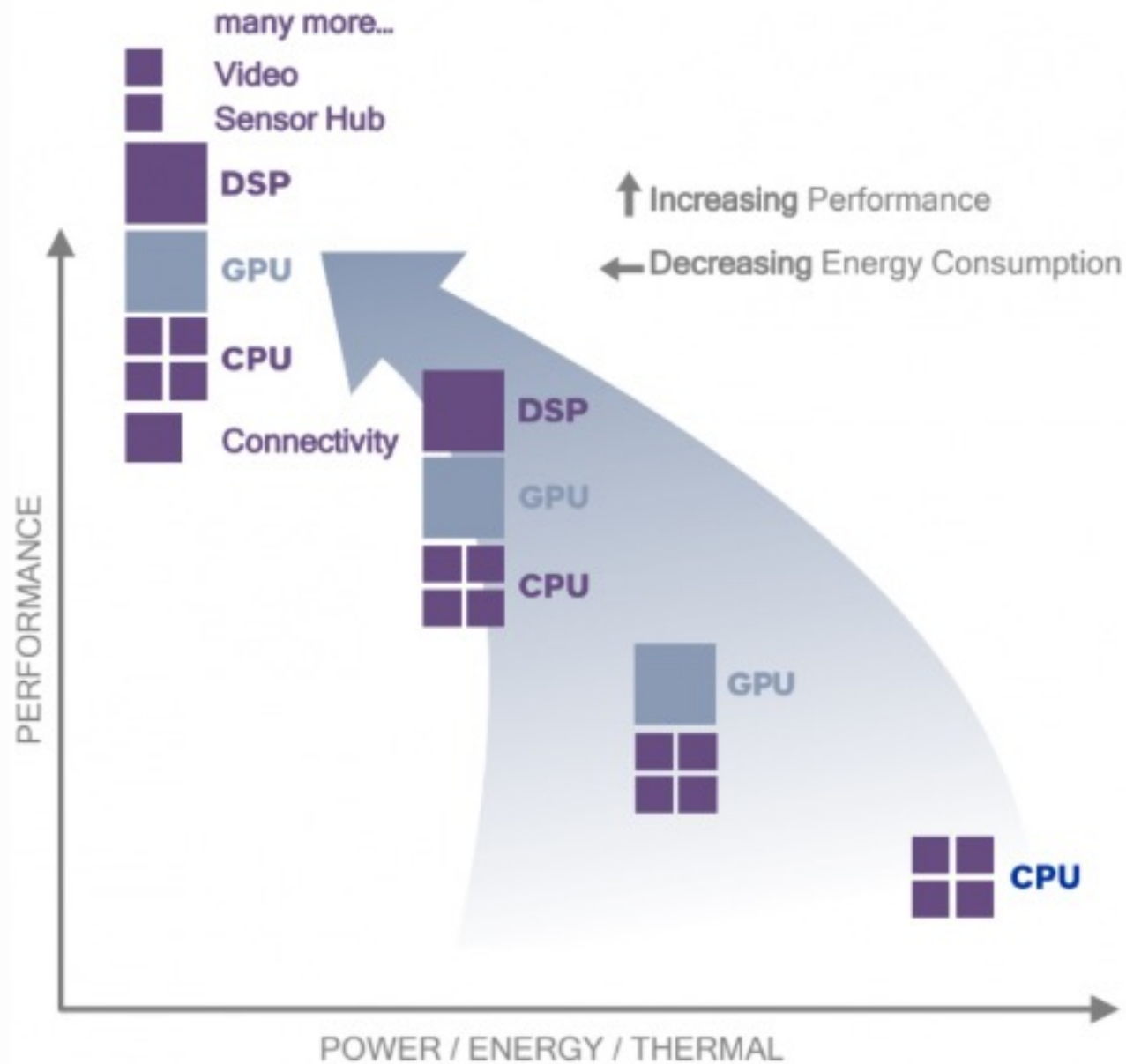


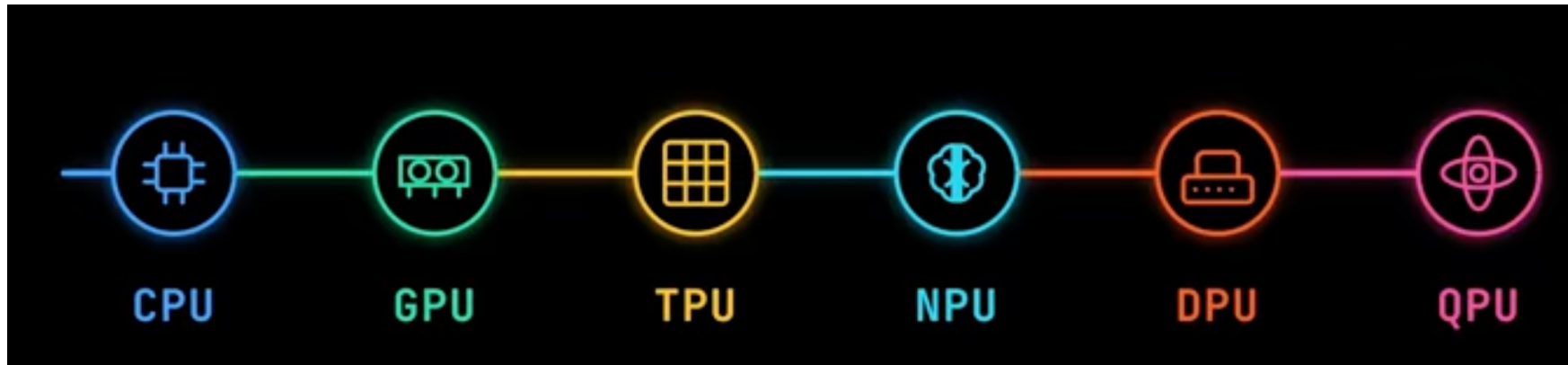
Fig 3 Amdahl's Law an Obstacle to Improved Performance Performance will not rise in the same proportion as the increase in CPU cores. Performance gains are limited by the ratio of software processing that must be executed sequentially. Amdahl's Law is a major obstacle in boosting multicore microprocessor performance. Diagram assumes no overhead in parallel processing. Years shown for design rules based on Intel planned and actual technology. Core count assumed to double for each rule generation.

Amdahl's Law

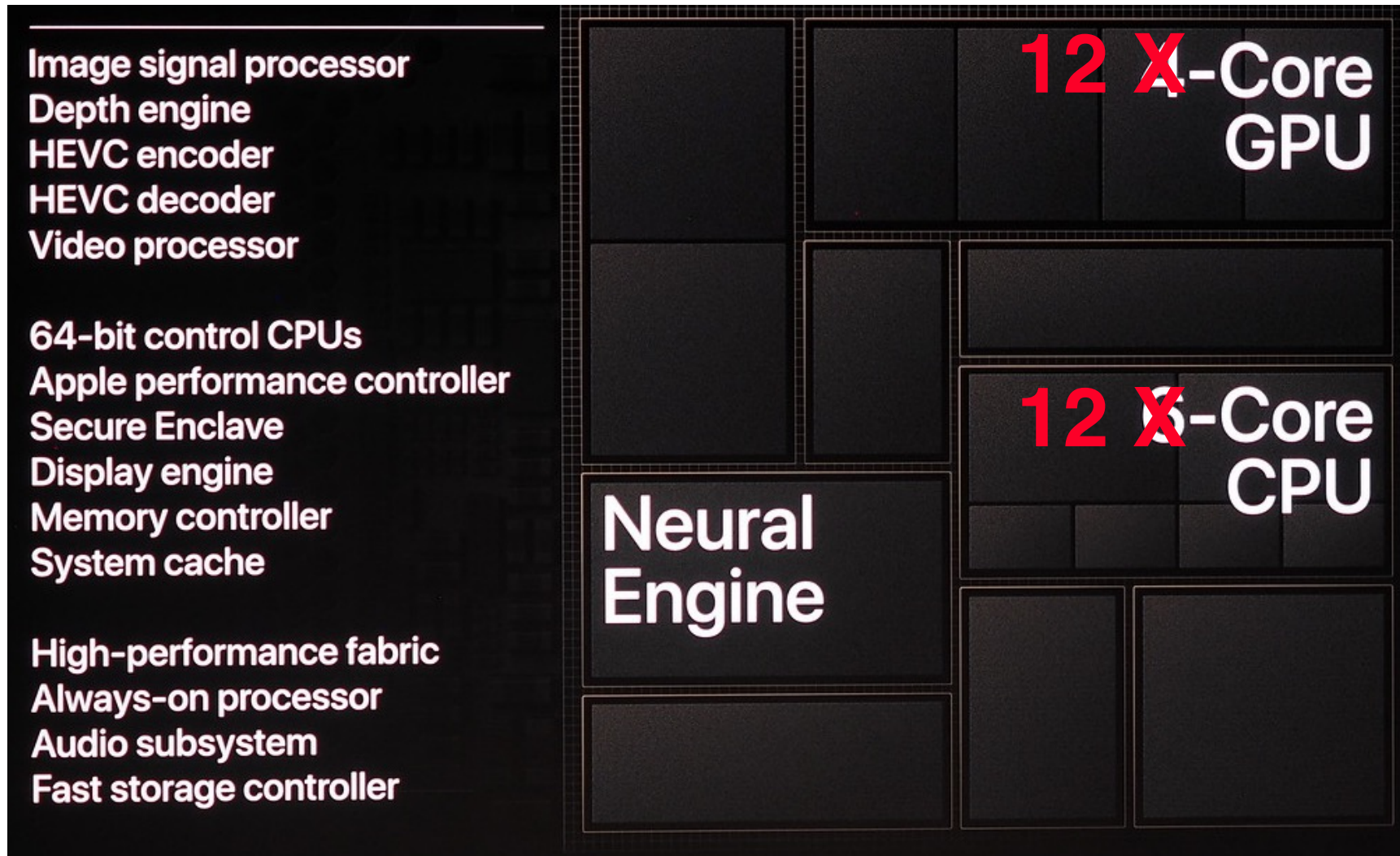




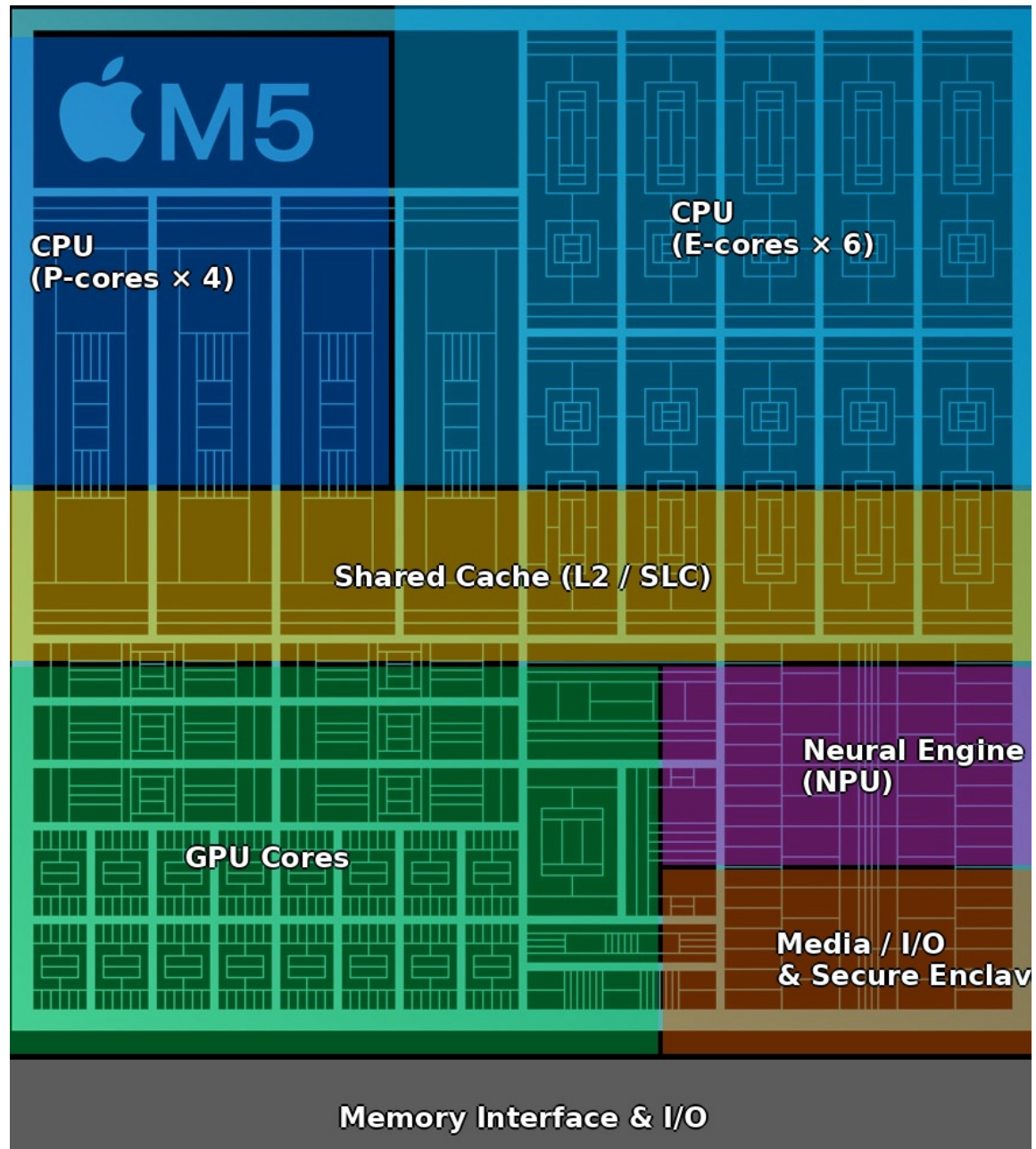
Advantages of heterogeneous computing. (Source: Qualcomm)

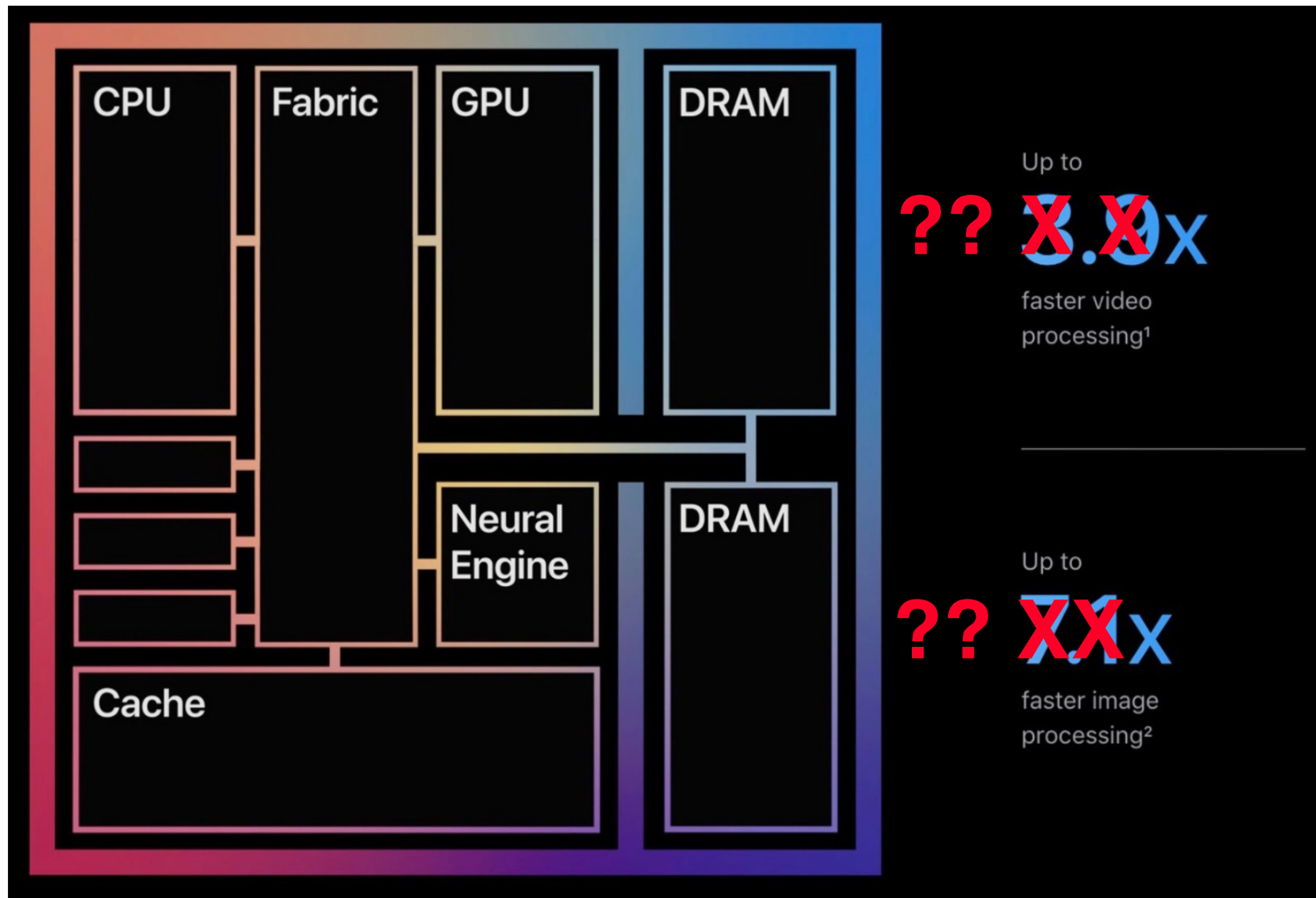


- **CPU** (**C**entral Processing Unit)
- **GPU** (**G**raphics Processing Unit)
- **TPU** (**T**ensor AI Processing Unit)
- **NPU** (**N**eural Processing Unit)
- **DPU** (**D**ata Processing Unit)
- **QPU** (**Q**uantum Processing Unit)



Apple's A7 Bionic. (Source: SLASHGEAR)





Apple's M1 Chip (Source: MACRUMORS)

CIS 570 Approach

- How are we going to address this change?
 - Start through single-processor systems
 - Study ILP and ways to exploit that
 - Delve into memory hierarchies for single processors
 - Talk about storage and I/O systems
 - We may touch on embedded systems at this point
 - Then, we'll look at multiprocessor systems
 - Discuss TLP and DLP
 - Talk about how multiprocessors affect memory design
 - Cover interconnection networks

Topic schedule

- Computer design fundamentals
- Basic ISA review
- Uniprocessor systems
 - Advanced pipelining—exploiting ILP & TLP
 - Memory hierarchy design
 - Storage & I/O (time permitting)
- Multiprocessor systems
 - Memory in multiprocessors
 - Synchronization (time permitting)

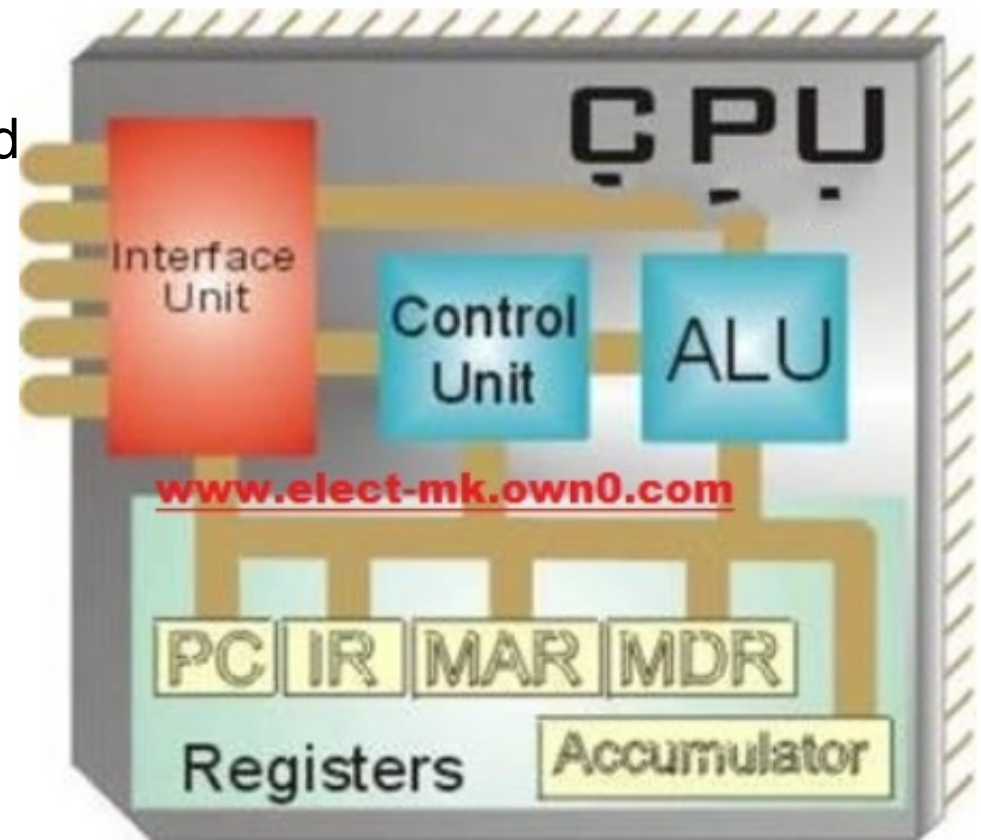
Design principles

1. Take advantage of **parallelism**
2. Principle of **locality**
3. Focus on the **common case**
4. **Amdahl's Law**
5. **Generalized processor performance**

COMPUTER EXECUTION CYCLE

Microprocessor

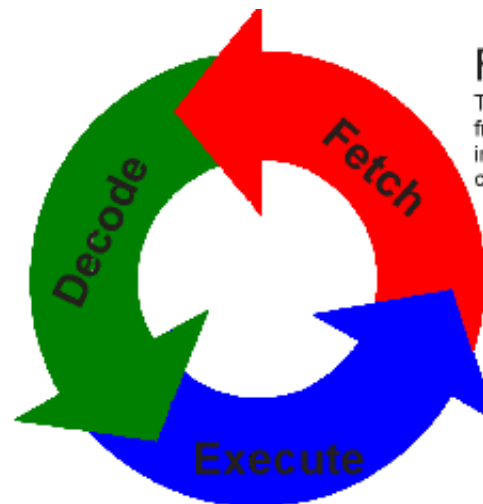
- **Arithmetic Logic Unit:** digital circuit that performs arithmetic and logical operations
- **Control Unit:** provides timing and control signals
- **Memory Address Register:** memory address of data to be read/written to RAM
- **Memory Data Register:** data to be written/read from RAM
- **Program Counter:** address of next instruction to be fetched
- **Instruction Register:** instruction currently being executed or decoded



Instruction Cycle

- **Fetch:** get instruction from memory
 - **Increment PC**
- **Decode:** translate instruction into a form that the CPU can understand (**microcode**) and **fetch operands** if needed
- **Execute:** perform the **action** as specified in the microcode

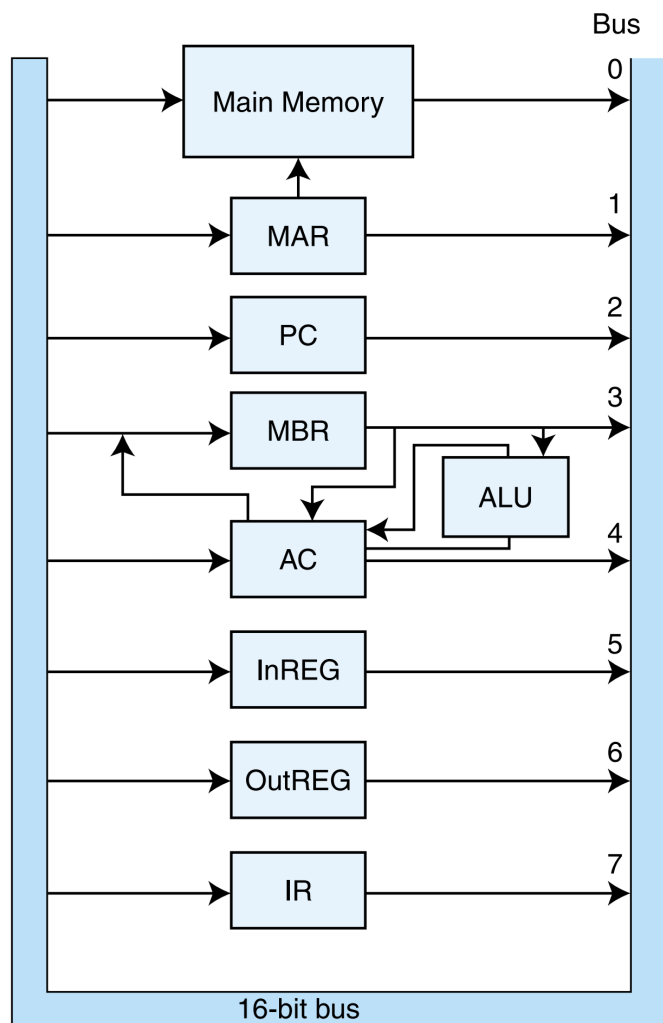
Decode:
the bit pattern into
the instruction
register



Fetch:
The next instruction
from memory and
increment the program
counter

Execute:
the action of the
instruction into the
instruction register

Data Path and Inst. Implementation



(MARIE: Machine Architecture
that is really Intuitive and Easy)

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

Load X : $MAR \leftarrow X$
 $MBR \leftarrow M[MAR], AC \leftarrow MBR$

Store X : $MAR \leftarrow X, MBR \leftarrow AC$
 $M[MAR] \leftarrow MBR$

Add X : $MAR \leftarrow X$
 $MBR \leftarrow M[MAR]$
 $AC \leftarrow AC + MBR$

Subt X : $MAR \leftarrow X$
 $MBR \leftarrow M[MAR]$
 $AC \leftarrow AC - MBR$

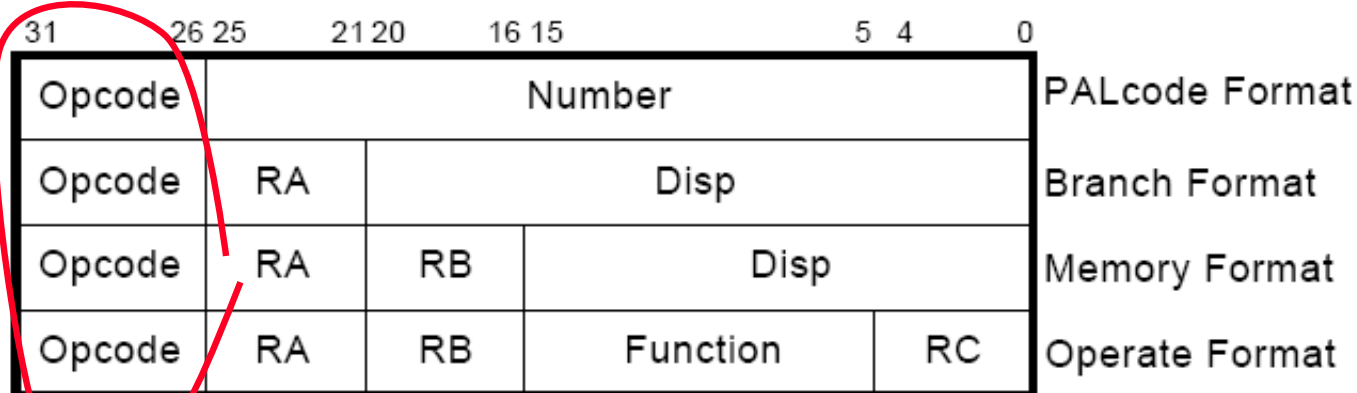
Input: $AC \leftarrow InREG$

Output: $OutREG \leftarrow AC$

INSTRUCTION SET ARCHITECTURE

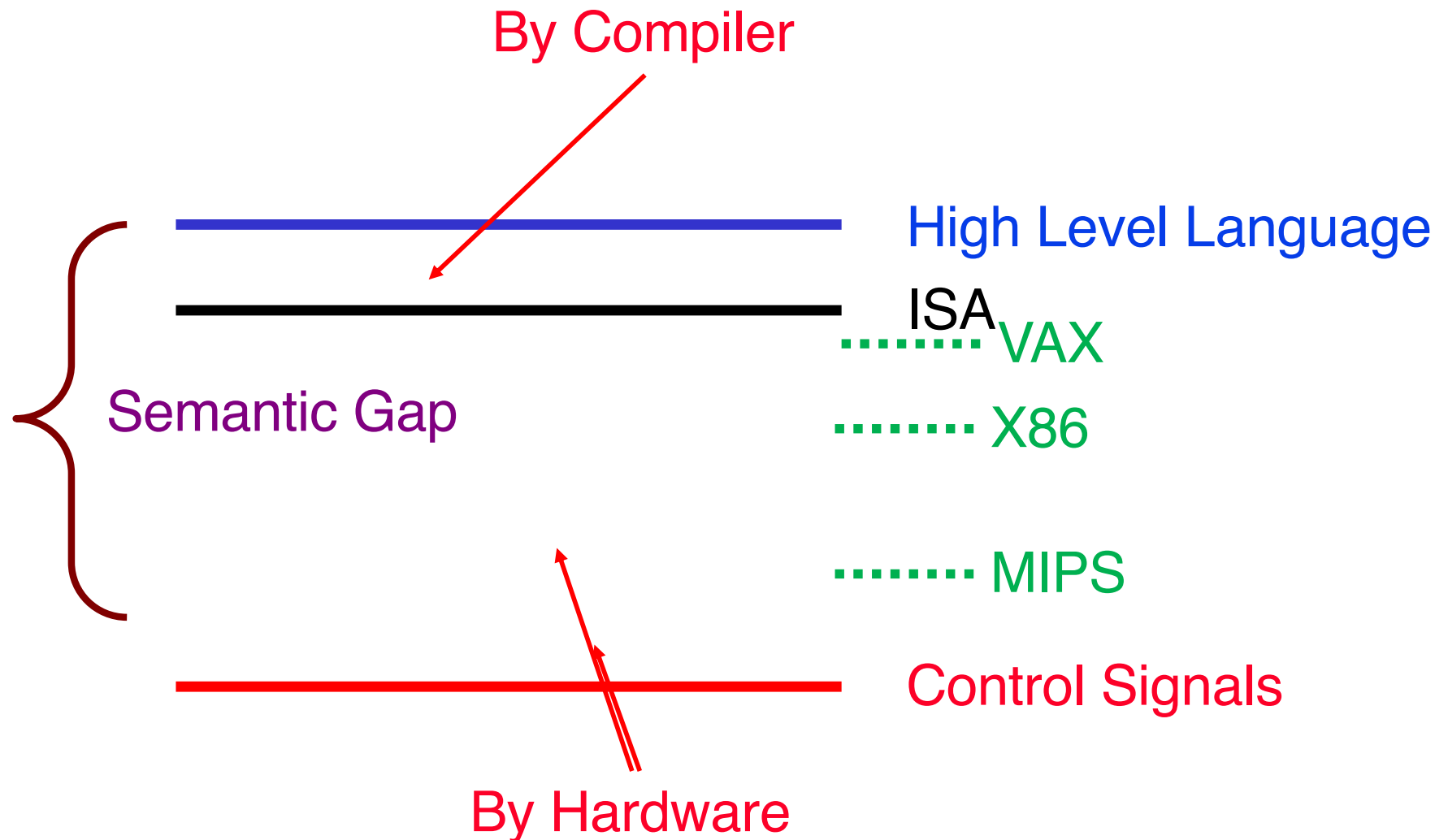
Elements of an Inst. Set Arch.

- Basic element of the HW/SW interface
- Consists of
 - **opcode**: what the instruction does
 - **operands**: who it is to do it to
 - Example from **Alpha ISA**:



Uniform Decode: same bits are used to represent the operation across different instruction format

Semantic Gap



RISC vs. CISC (cont.)

CISC	RISC
Emphasis on hardware	Emphasis on software
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
Small code sizes, high cycles per second	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{time}}{\text{cycle}}$$

CISC: minimizes the number of instructions per program at cost of number of cycles per instruction

RISC: reduces the cycles per instruction at the cost of the number of instructions per program

MIPS: **M**icroprocessor without **I**nterlocked **P**ipelined **S**tages

MIPS ISA

MIPS ISA

A fast instructions is easier to decode

Type	-31-	format (bits)					-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)	
I	opcode (6)	rs (5)	rt (5)	immediate (16)			
J	opcode (6)	address (26)					

An easy to decode Inst is simple one

Simplicity favors regularity

Decoding Example (1/7)

- Here are six machine language instructions in hexadecimal:

00001025_{hex}

0005402A_{hex}

11000003_{hex}

00441020_{hex}

20A5FFFF_{hex}

08100001_{hex}

- Let the first instruction be at address 4,194,304_{ten} (0x00400000_{hex}).
- Next step: convert hex to binary

Decoding Example (2/7)

- The six machine language instructions in binary:

```

00000000000000000000000010000000100101
000000000000000010101000000000101010
000100010000000000000000000000000011
0000000000100010000010000000100000
00100000101001011111111111111111
0000100000001000000000000000000001
  
```

Next step: identify opcode and format

R	0	rs	rt	rd	shamt	funct
I	1, 4-31	rs	rt	immediate		
J	2 or 3	target address				

Decoding Example (3/7)

- Select the opcode (first 6 bits) to determine the format:

R	000000	000000	000000	00010	000000	100101
R	000000	000000	00101	01000	000000	101010
I	000100	010000	000000	000000	000000	000011
R	000000	00010	00100	00010	000000	100000
I	001000	00101	00101	11111	11111	111111
J	000010	000001	000000	000000	000000	000001

- Look at **opcode**:
 0 means R-Format,
 2 or 3 mean J-Format,
 otherwise I-Format.
- Next step: **separation of fields**

Decoding Example (4/7)

- Fields separated based on format/opcode:

Format:

R	000000	000000	000000	00010	000000	100101
R	000000	000000	00101	01000	000000	101010
I	000100	01000	000000	000000000000000011		
R	000000	00010	00100	00010	000000	100000
I	001000	00101	00101	1111111111111111		
J	000010	00000010000000000000000001				

- Next step: translate (“disassemble”) to MIPS assembly instructions

Decoding Example (4/7)

- Fields separated based on format/opcode:

Format:

R	0	0	0	2	0	37
R	0	0	5	8	0	42
I	4	8	0	+3		
R	0	2	4	2	0	32
I	8	5	5	-1		
J	2	1,048,577				

- Next step: translate (“disassemble”) to MIPS assembly instructions

Decoding Example (5/7)

- MIPS Assembly (Part 1):

Address	Assembly instructions
0x00400000	or \$2, \$0, \$0
0x00400004	slt \$8, \$0, \$5
0x00400008	beq \$8, \$0, 3
0x0040000c	add \$2, \$2, \$4
0x00400010	addi \$5, \$5, -1
0x00400014	j 0x100001

- **Better solution:** translate to more meaningful MIPS instructions (fix the branch/jump and add labels, registers)

Decoding Example (6/7)

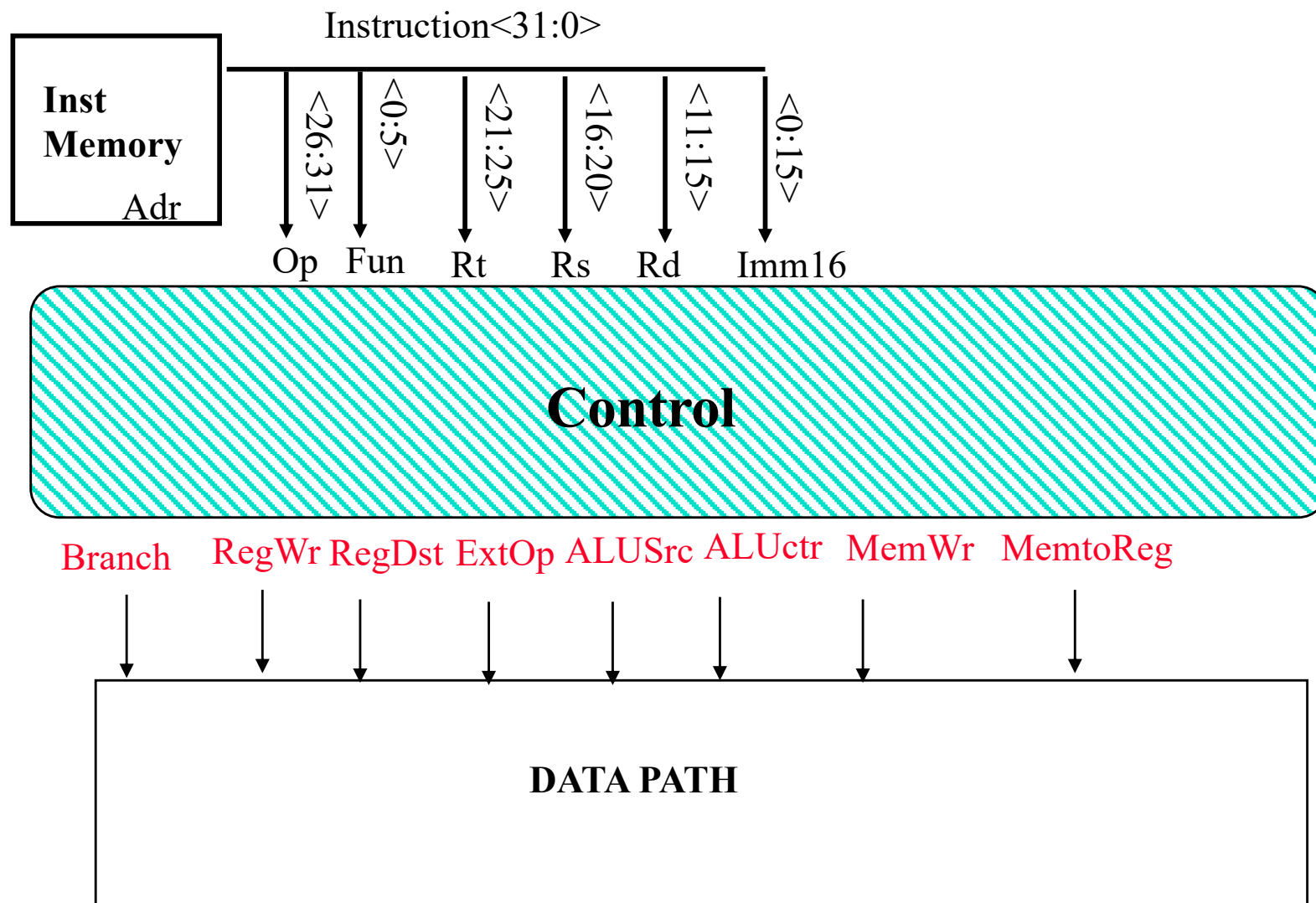
- MIPS Assembly (Part 2):

```
                or      $v0,$0,$0
Loop:          slt      $t0,$0,$a1
                beq      $t0,$0,Exit
                add      $v0,$v0,$a0
                addi     $a1,$a1,-1
                j         Loop
Exit:
```

- Next step: translate to C code (must be creative!)

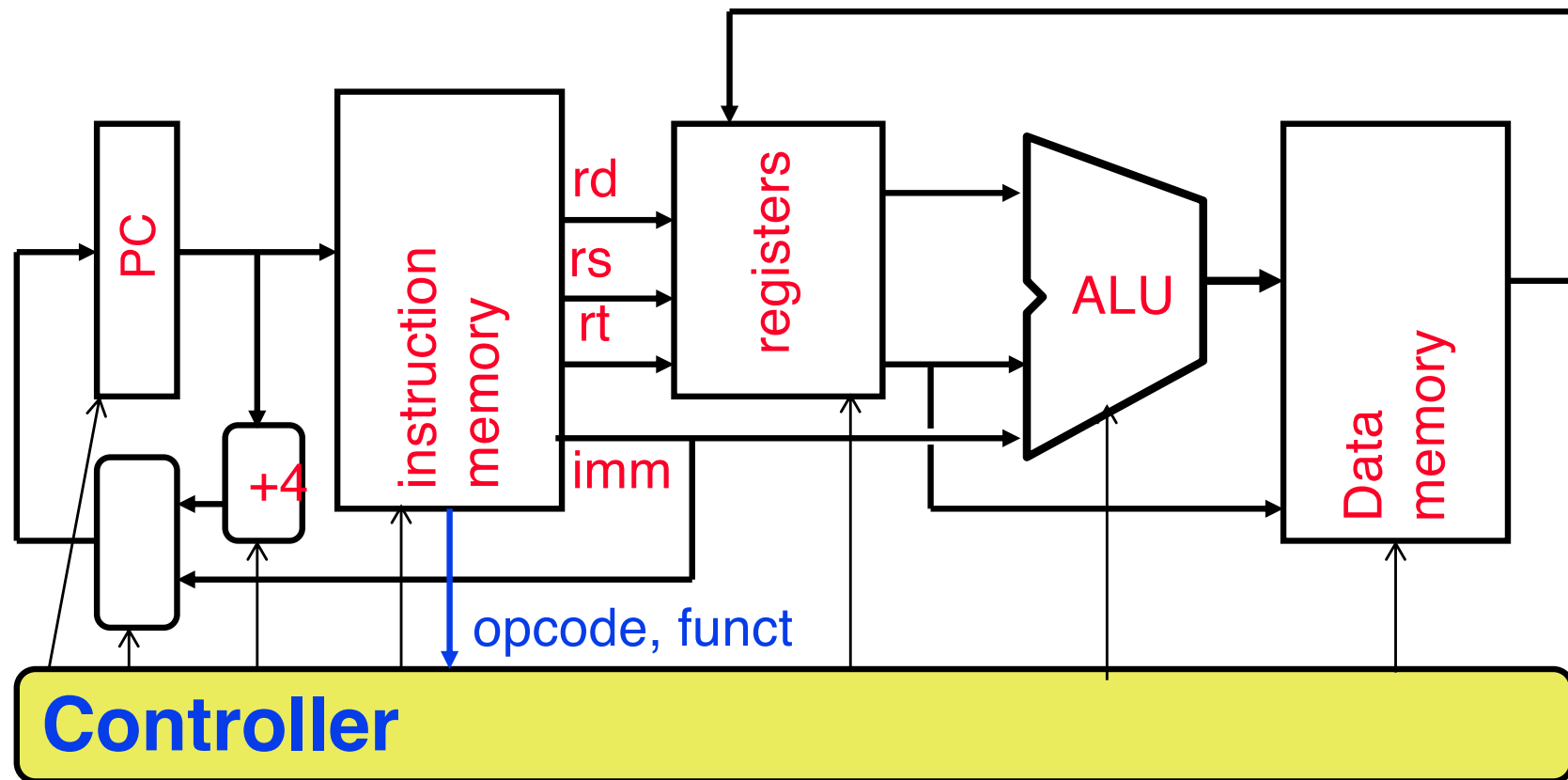
IMPLEMENTING AN ISA

Control and Data Paths



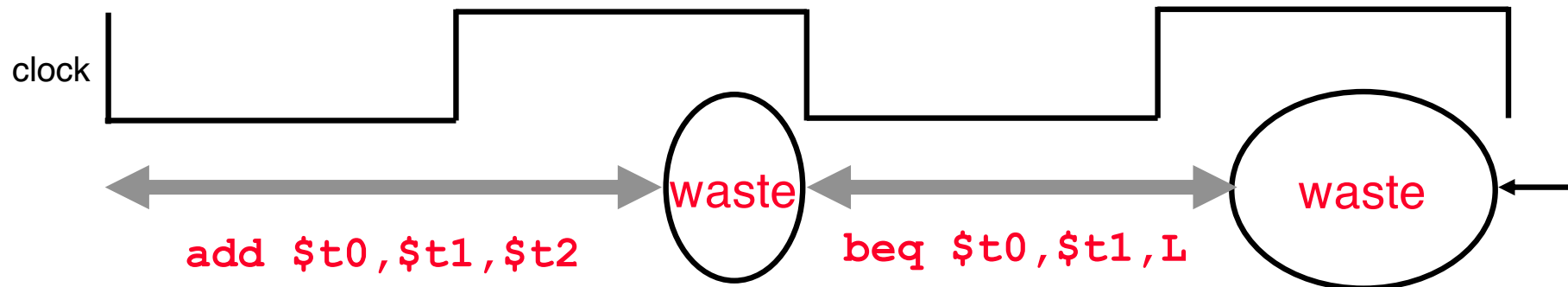
Datapath Summary

- The datapath based on data transfers required to perform instructions
- A controller causes the right transfers to happen

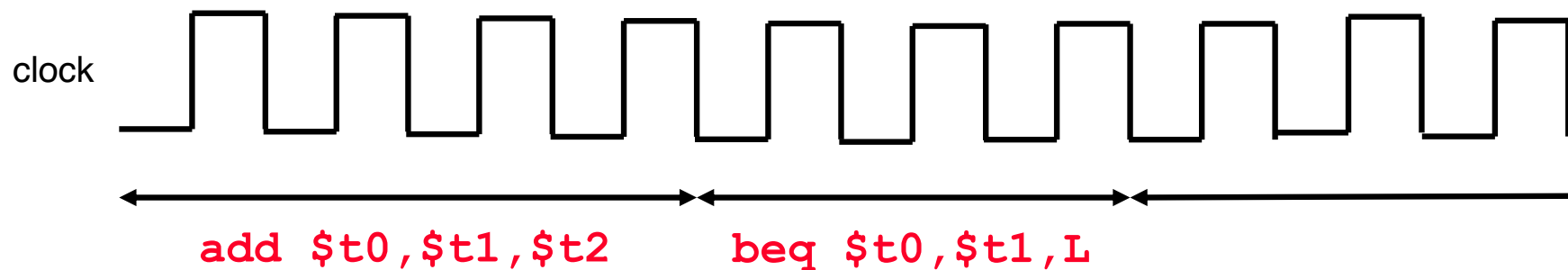


Recap: Clocking: Single-cycle vs. Multicycle

Single-cycle Implementation



Multicycle Implementation



- Multi cycle Imp.: **less waste = higher performance**

Instruction Level Parallelism (ILP)

PIPELINING

Performance Measures

- Latency (execution time)
 - Time to finish a task
- Throughput
 - Total work done per unit time
 - e.g., tasks/transactions/... per hour
- How are latency and throughput affected by
 - Replacing the processor with a faster version?
 - Adding more processors?

5-Stage Pipeline Diagram

	Cycle							
	1	2	3	4	5	6	7	8
LW	IF	ID	EX	MEM	WB			
DADD		IF	ID	EX	MEM	WB		
BEQ			IF	ID	EX	MEM	WB	
SW				IF	ID	EX	MEM	WB

- **Pipeline diagram** shows execution of multiple instructions
- MIPS has five pipeline stages
 - **IF**: Instruction fetch
 - **ID**: Instruction decode
 - **EX**: Execution / address calculation
 - **MEM**: Memory access
 - **WB**: Write back result to register

Sample Program

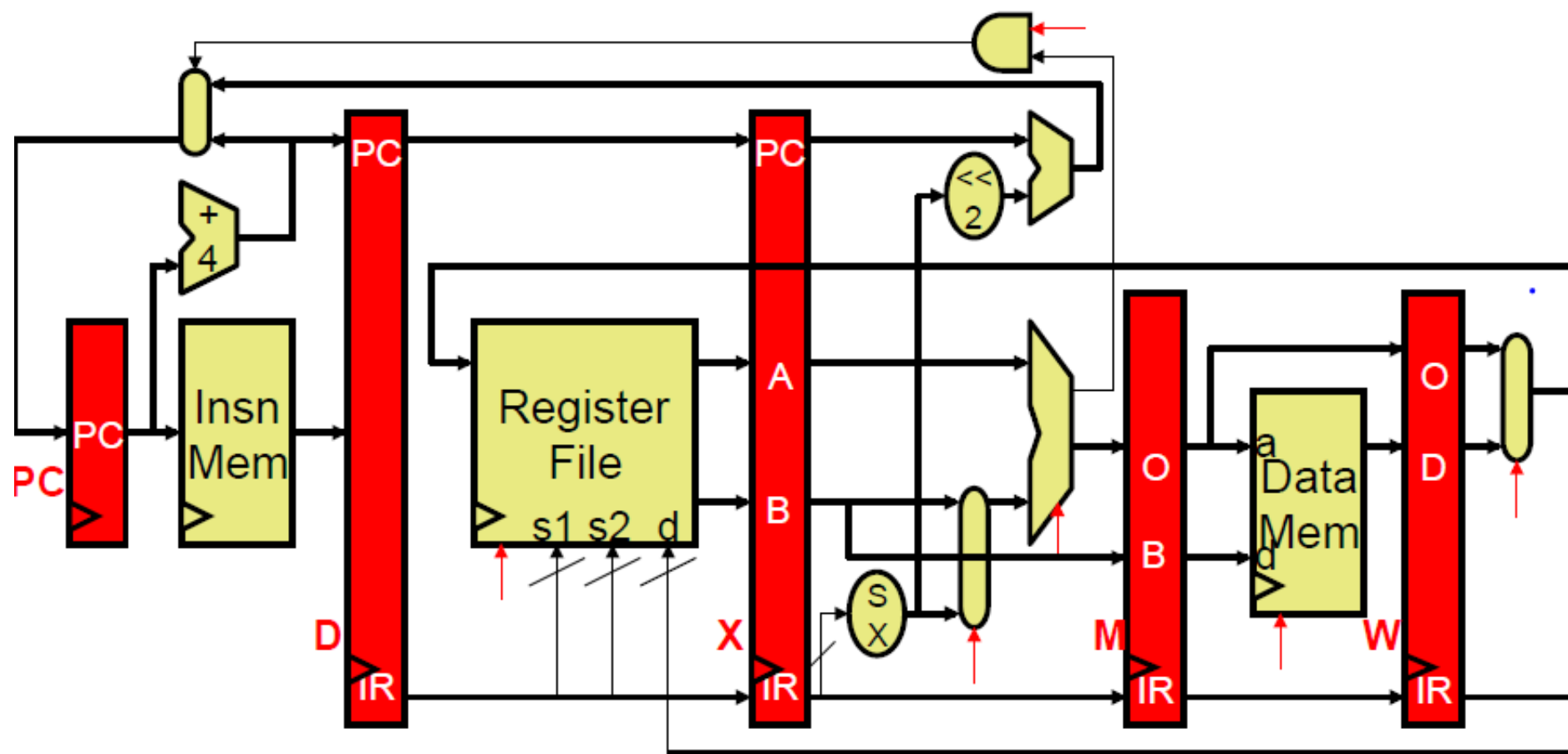
add \$3,\$2,\$1

lw \$4,8(\$5)

sw \$6,4(\$7)

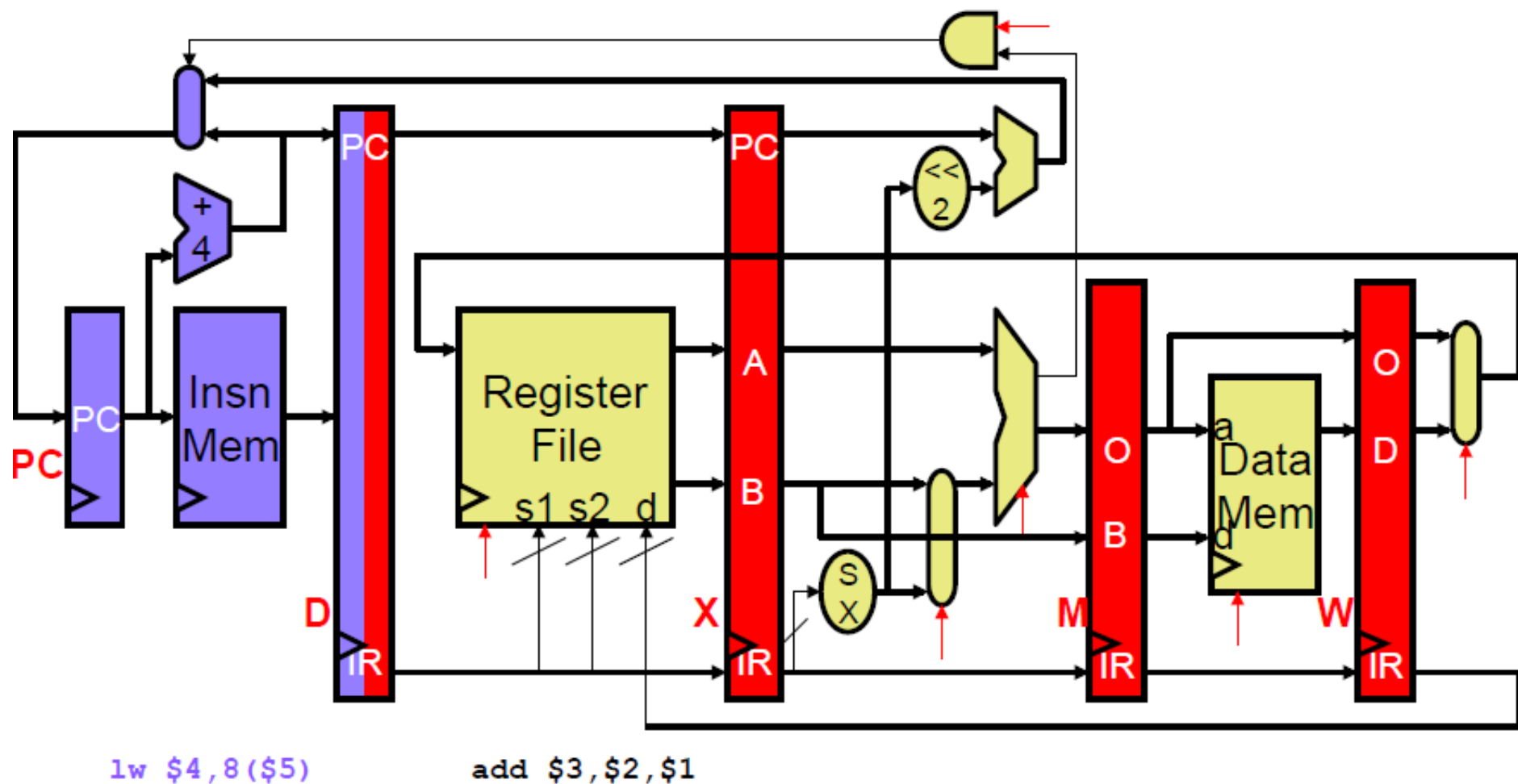
Cycle							
	1	2	3	4	5	6	7
Add.	IF	ID	EX	MEM	WB		
Lw .		IF	ID	EX	MEM	WB	
Sw .			IF	ID	EX	MEM	WB

Pipeline Example: Cycle 1

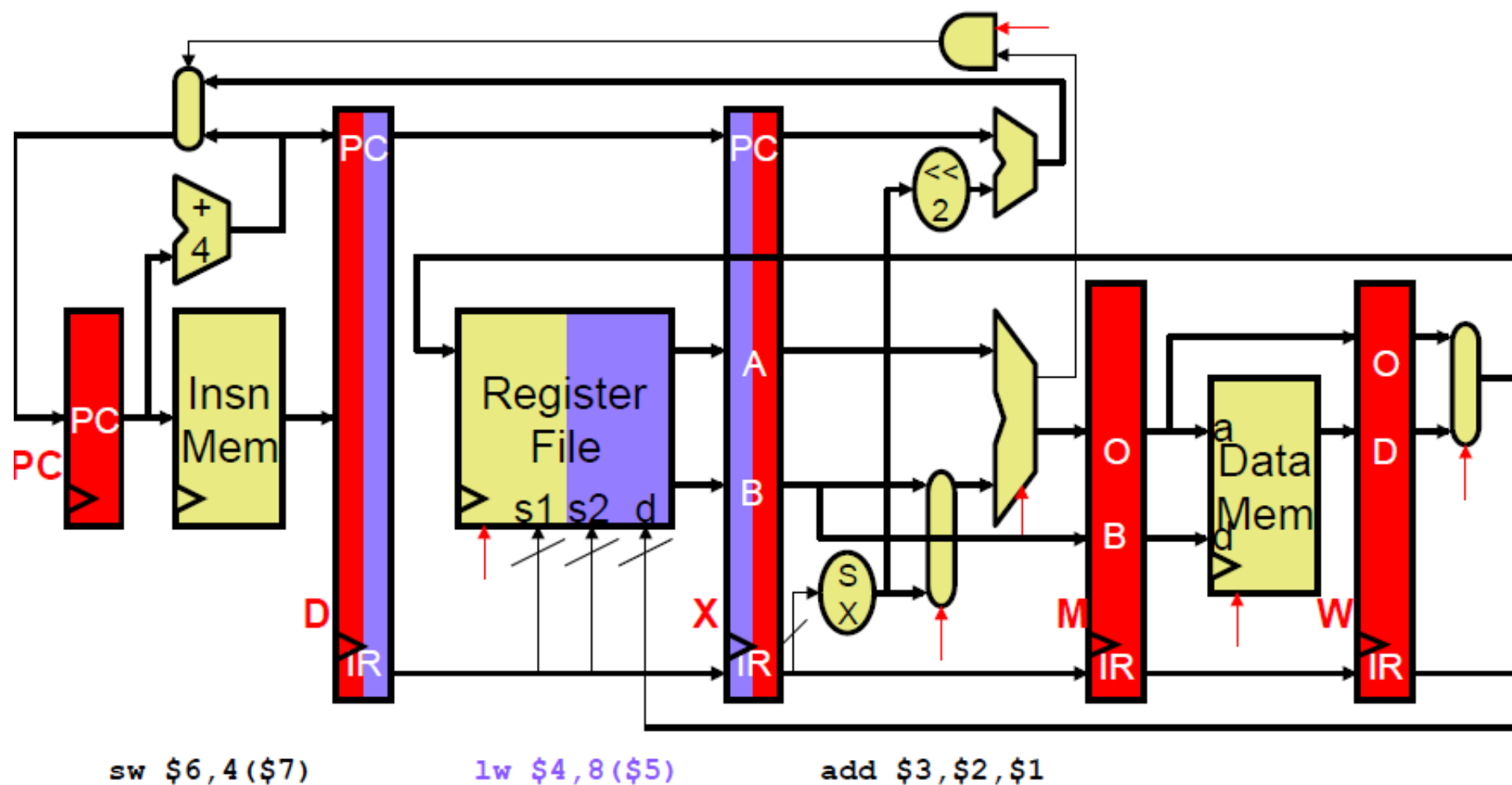


add \$3,\$2,\$1

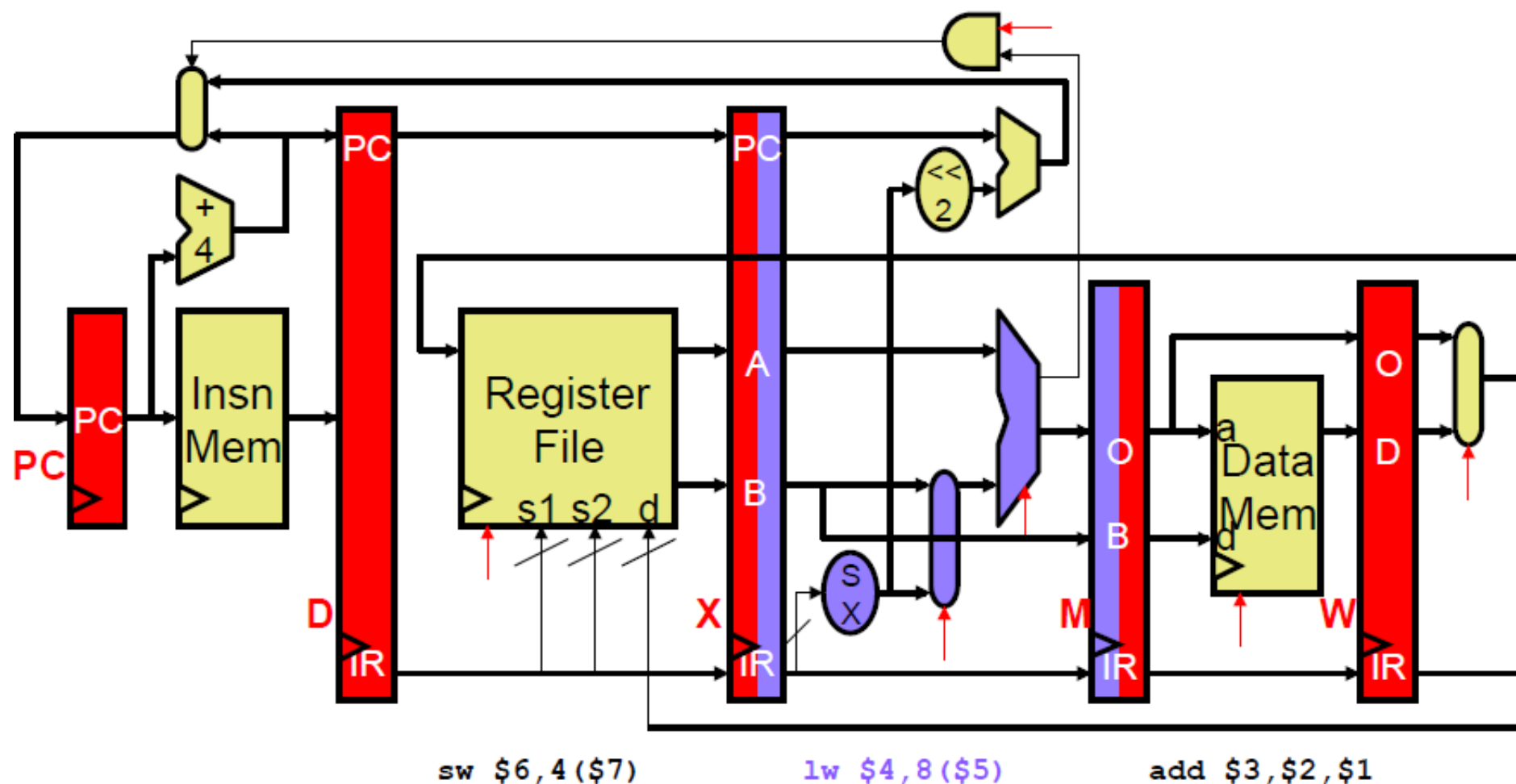
Pipeline Example: Cycle 2



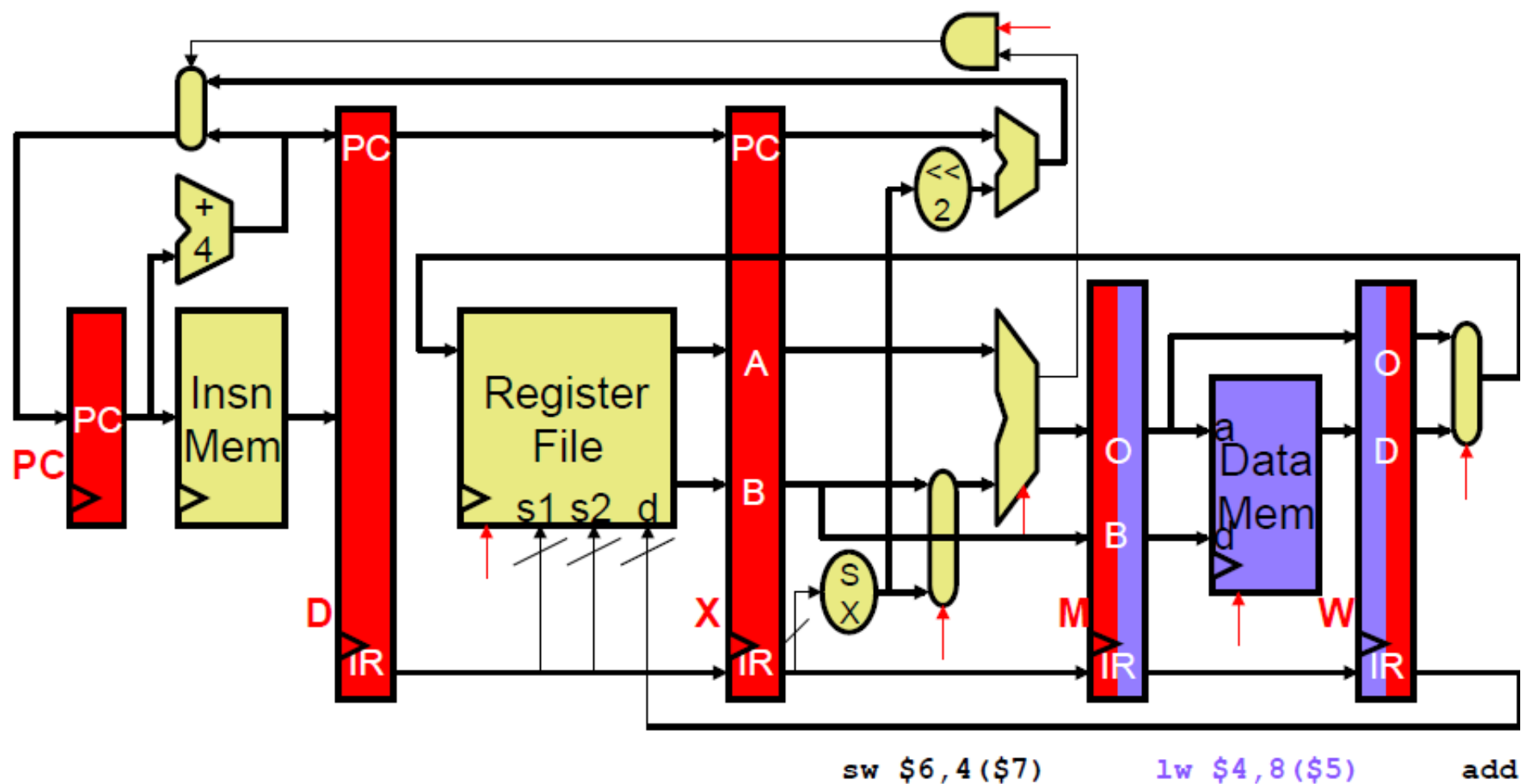
Pipeline Example: Cycle 3



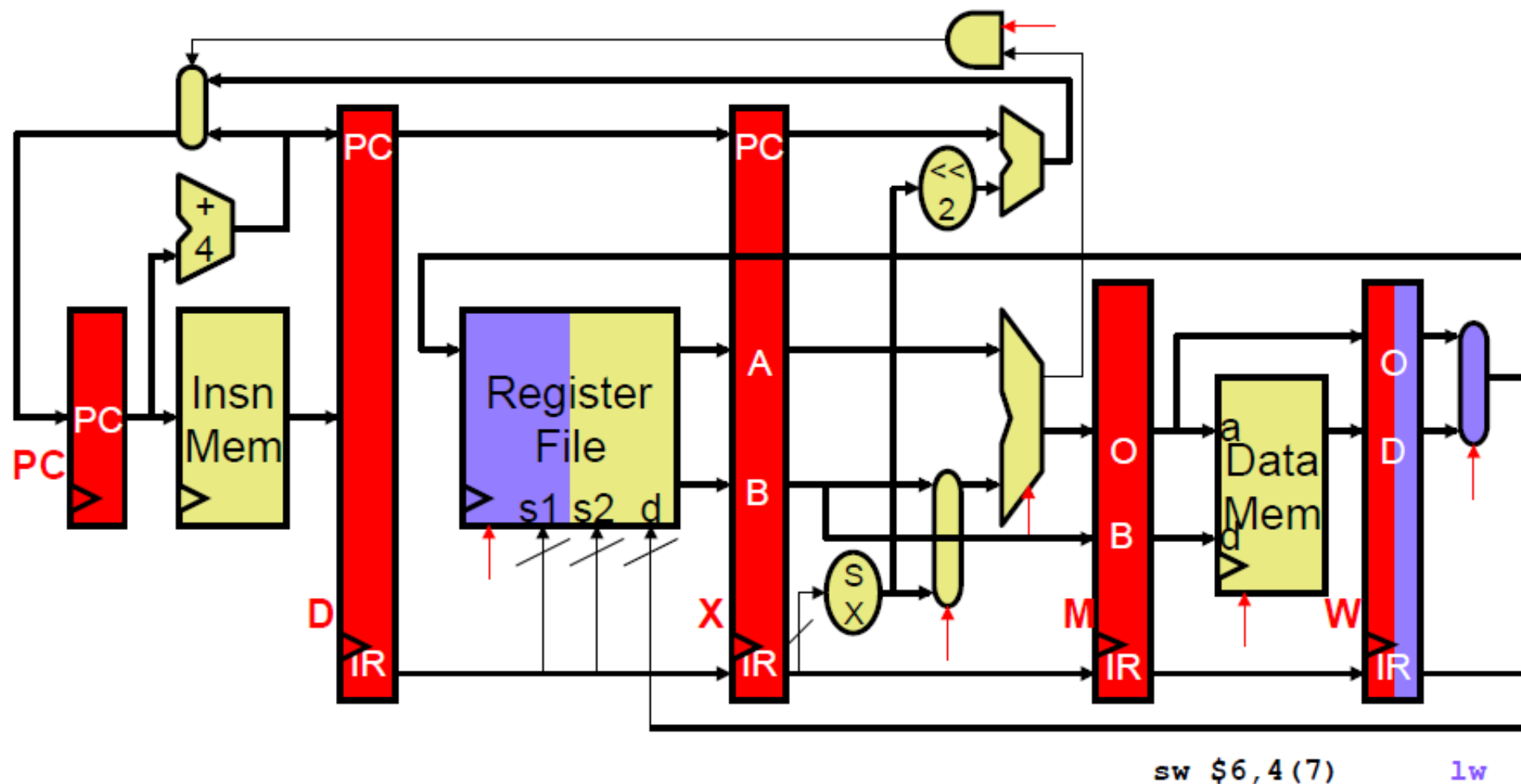
Pipeline Example: Cycle 4



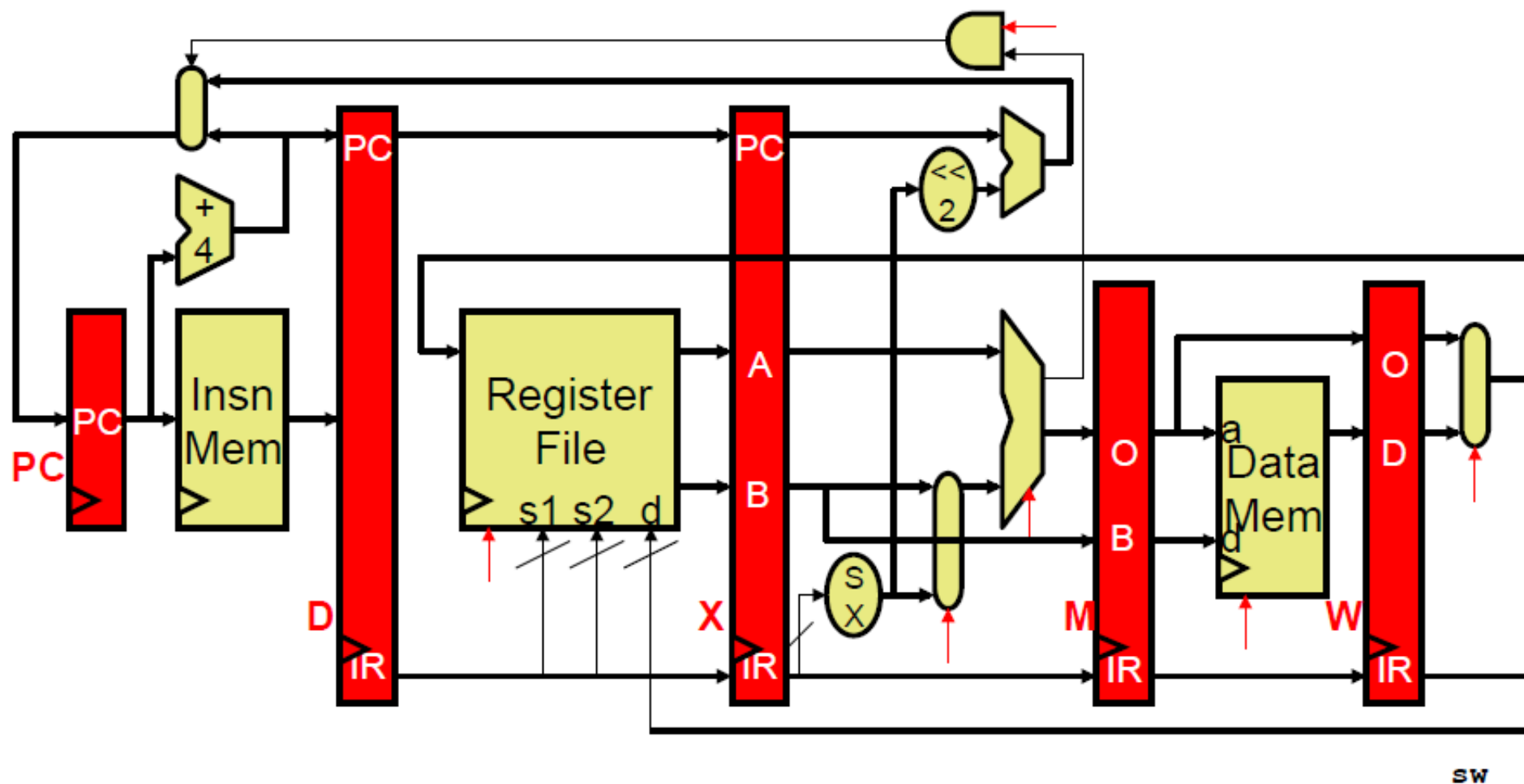
Pipeline Example: Cycle 5



Pipeline Example: Cycle 6



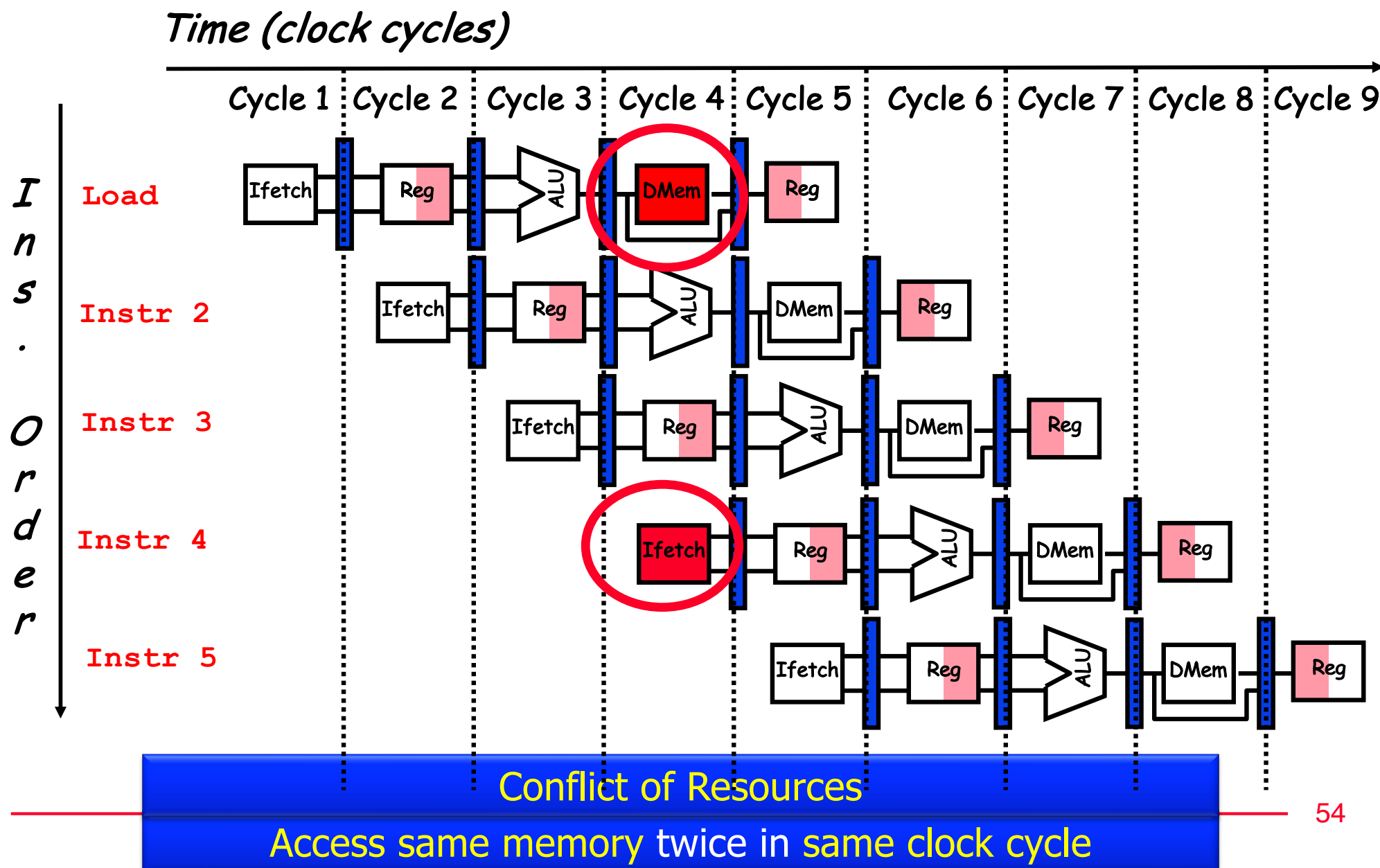
Pipeline Example: Cycle 7



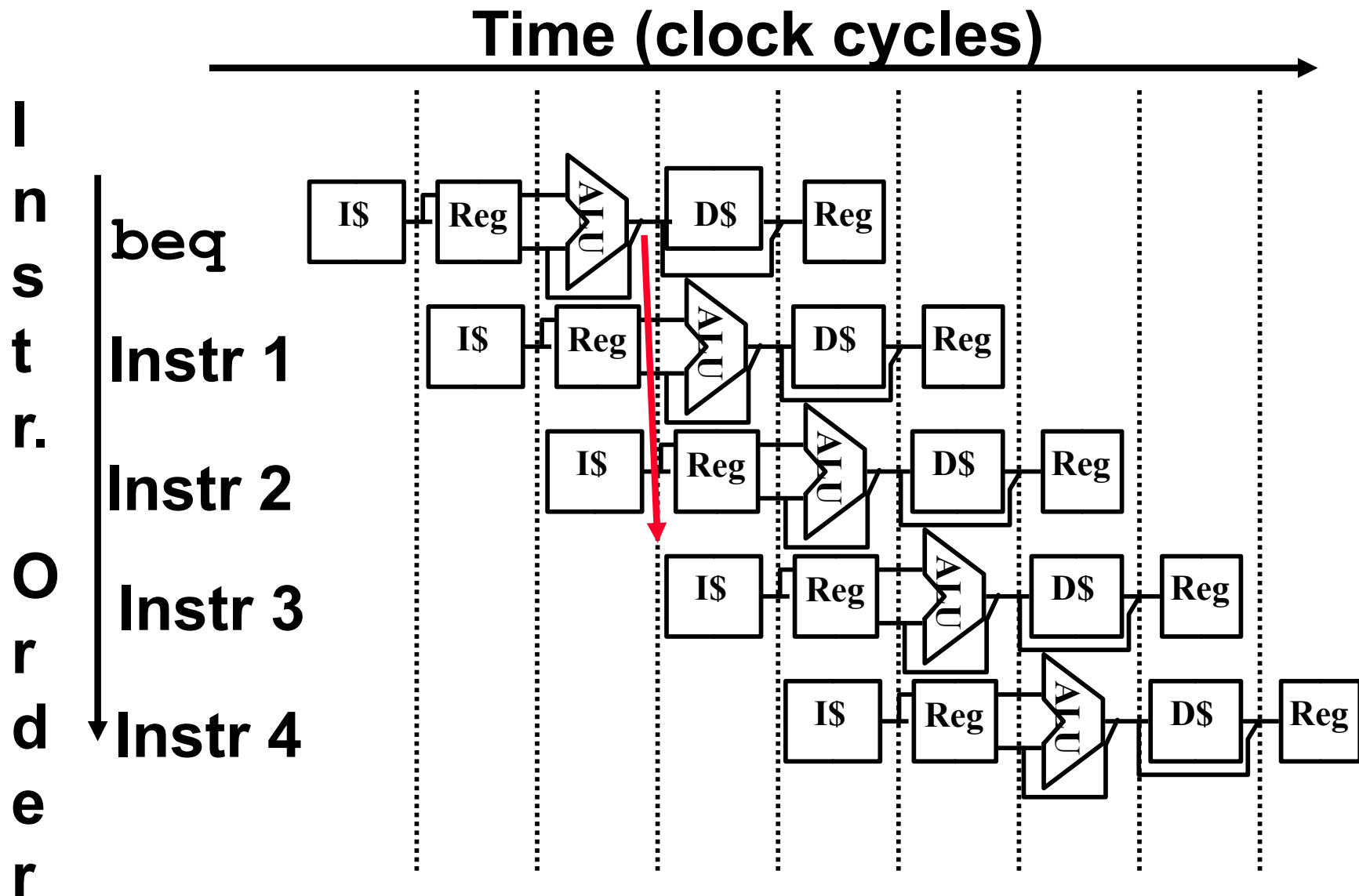
Pipelining is not quite that easy!

- Limits to **pipelining**: Hazards **prevent** next **instruction** from **executing** during its designated clock cycle
 - **Structural hazards**: HW cannot support this combination of instructions
 - Multiple **instructions trying to use same hardware**
 - **Data hazards**: Instruction **depends on result of prior instruction** still in the pipeline
 - **Control hazards**: Caused by delay between the fetching of instructions and decisions about **changes in control flow** (**branches** and **jumps**).

Structural Hazard #1: Single Memory (1/3)



Control Hazard: Branching (1/8)



Where is comparison for the branch done?

Data Hazard on R1

*I
n
s
t
r.

O
r
d
e
r*

add **r1**, r2, **r3**

sub r4, **r1**, **r3**

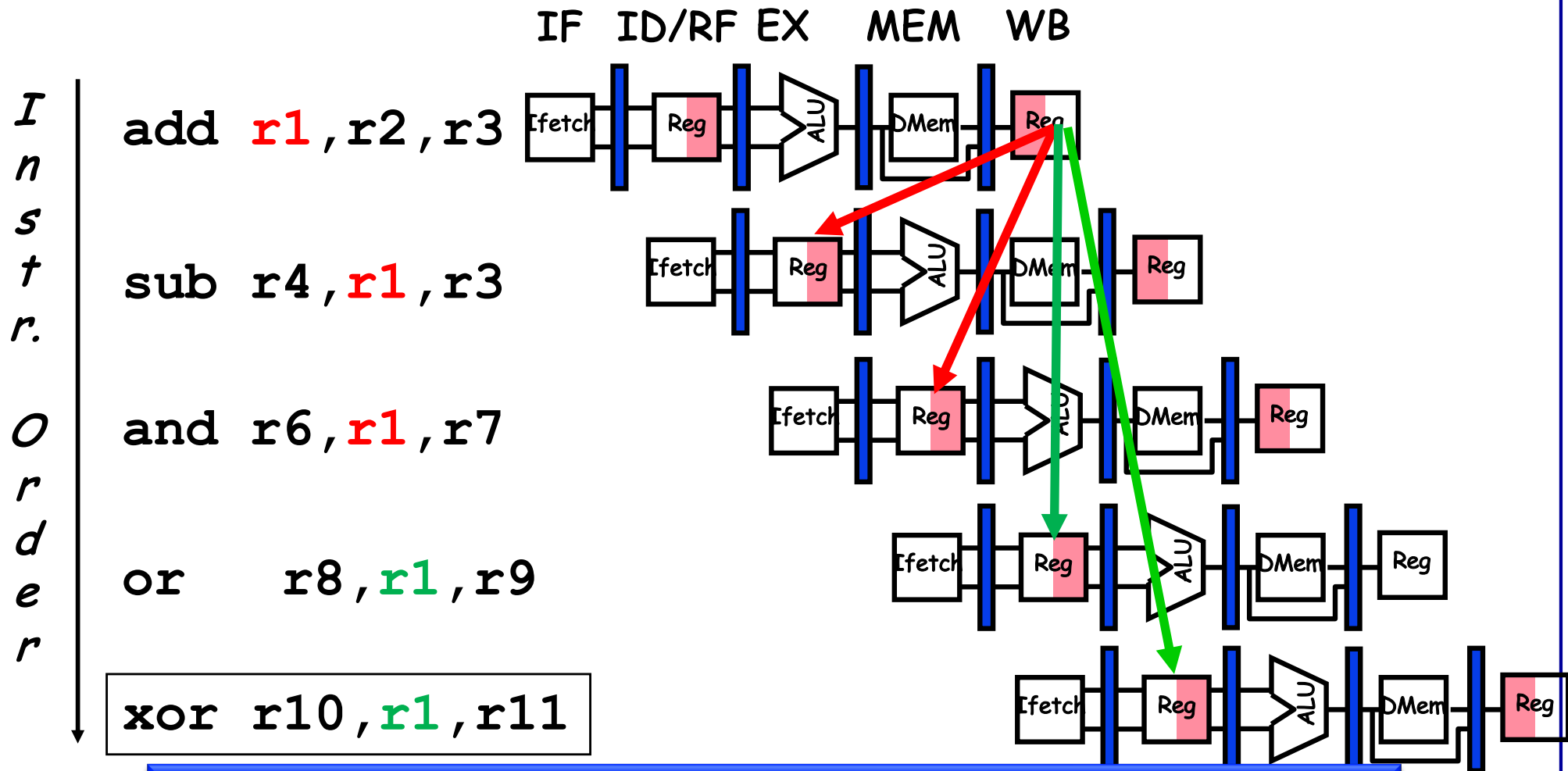
and r6, **r1**, r7

or r8, **r1**, r9

xor r10, **r1**, r11

Data Hazard on R1

Time (clock cycles)



Data-flow backward in time are hazards

ILP: LOOP UNROLLING

Instruction Level Parallelism

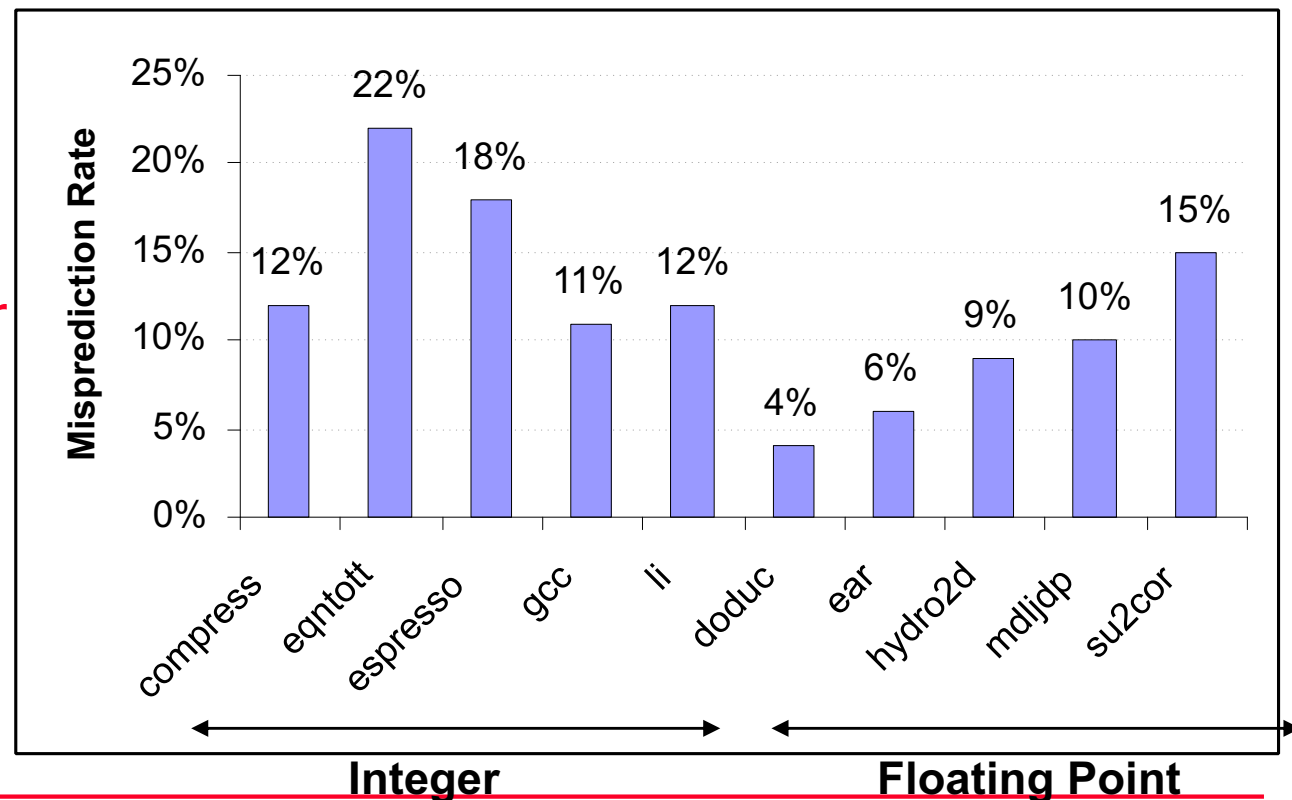
- Instruction-Level Parallelism (ILP): the ability to overlap instruction execution
- Two approaches to exploit ILP & improve performance:
 1. Use hardware to dynamically find parallelism while running program
 2. Use software to find parallelism statically at compile-time

ILP: BRANCH PREDICTION

Static Branch Prediction

- We scheduled code around delayed branch
- To **reorder code around branches**, need to **predict branch statically** during **compilation**
- Simplest scheme is to **predict a branch as taken**
 - Average misprediction = untaken branch frequency = 34% SPEC

• More accurate scheme predicts branches using profile information collected from earlier runs, and modify prediction based on last run



Dynamic Branch Prediction

- Why does prediction work?
 - Underlying **algorithm** has **regularities**
 - **Data** that is being operated on has **regularities**
 - **Instruction sequence** has **redundancies** that are artifacts of way that humans/compiler think about problems
- Is **dynamic branch** prediction **better** than **static branch prediction**?
 - Seems to be
 - There are a small number of **important branches** in programs which have **dynamic behavior**

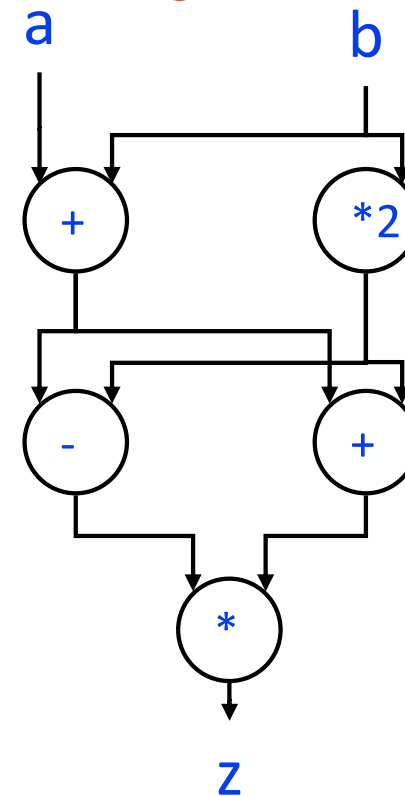
ILP: DYNAMIC SCHEDULING

Von Neumann vs Dataflow

- Consider a Von Neumann program
 - What is the **significance** of the **program order**?
 - What is the **significance** of the **storage locations**?

```
v <= a + b;  
w <= b * 2;  
x <= v - w  
y <= v + w  
z <= x * y
```

Sequential



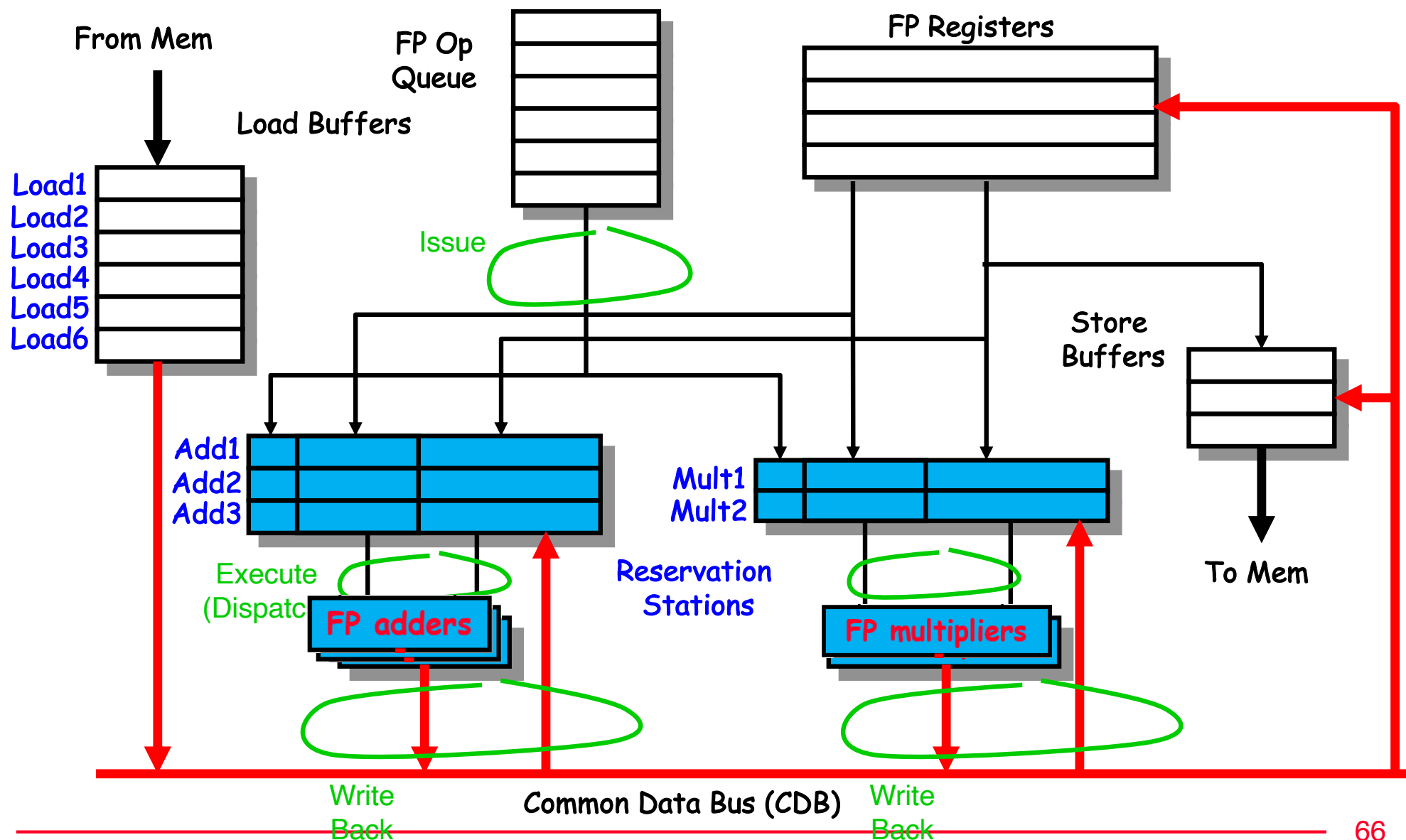
Dataflow

- Which model is more natural to you as a programmer?

Tomasulo's Algorithm

- Control & buffers **distributed** with **Function Units (FU)**
 - **FU buffers** called “reservation stations(RS)”; have pending operands
- Registers in instructions replaced by values or pointers to reservation stations(RS); called register renaming
 - **Renaming**: **avoids WAR, WAW hazards**
 - **More reservation stations than registers**, so can perform optimizations compilers can't
- **Results to FU from RS**, **not through registers**, over Common Data Bus that **broadcasts** results to all FUs
 - **Forwarding**: **avoids RAW** hazards by **executing an instruction only when its operands are available**
- **Loads and Stores** are treated as **FUs with RSs as well**
- Integer instructions can go past branches (**predict taken**), allowing FP ops beyond basic block in FP queue

Tomasulo Organization



ILP: SPECULATION

Speculation to greater ILP

- 3 components of **HW-based speculation**:
 1. **Dynamic branch prediction** to choose which instructions to execute
 2. **Speculation** to allow execution of instructions before control dependences are resolved
 - Add **ability to undo effects** of **incorrectly speculated** sequence
 3. **Dynamic scheduling** to deal with **scheduling of different combinations of basic blocks**

THREAD LEVEL PARALLELISM (TLP)



DATA LEVEL PARALLELISM

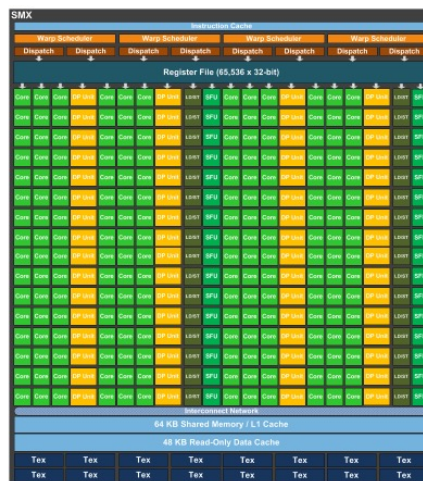
SIMD Processing

- Single instruction operates on multiple data elements
 - In time or in space
- Multiple Processing Elements (Functional Units)
- Time-space duality
 - Array processor: Instruction operates on multiple data elements at the same time
 - Vector processor: Instruction operates on multiple data elements in consecutive time steps

DOMAIN SPECIFIC ARCHITECTURE

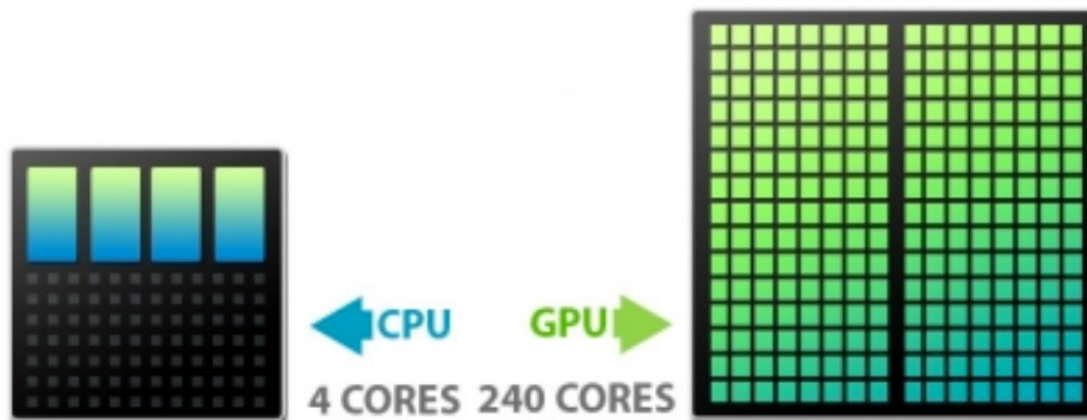
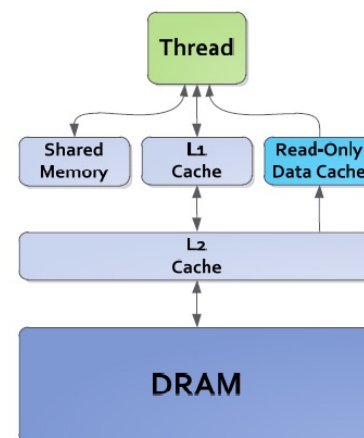
Many Core GPU Processors

Massive Parallelism ($\sim 5k$ cores)



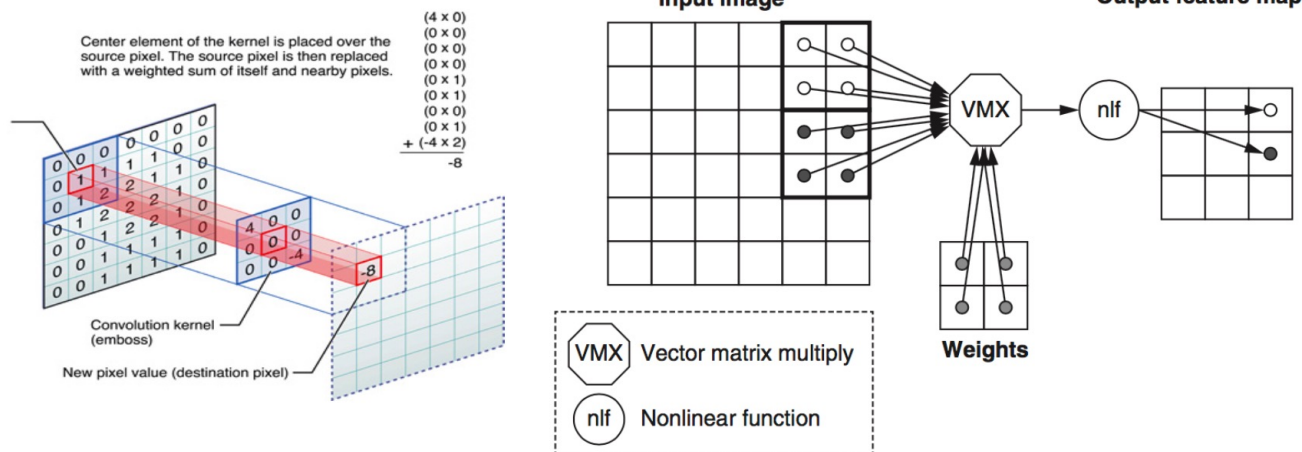
SMX: 192 single-precision CUDA cores, 64 double-precision units, 32 special function units (SFU), and 32 load/store units (LSU/STU).

Kepler Memory Hierarchy



Convolutional Neural Network

- Computer vision
- Each layer raises the level of abstraction
 - First layer recognizes horizontal and vertical lines
 - Second layer recognizes corners
 - Third layer recognizes shapes
 - Fourth layer recognizes features (e.g. ears of a dog)
 - Higher layers recognizes different breeds of dogs

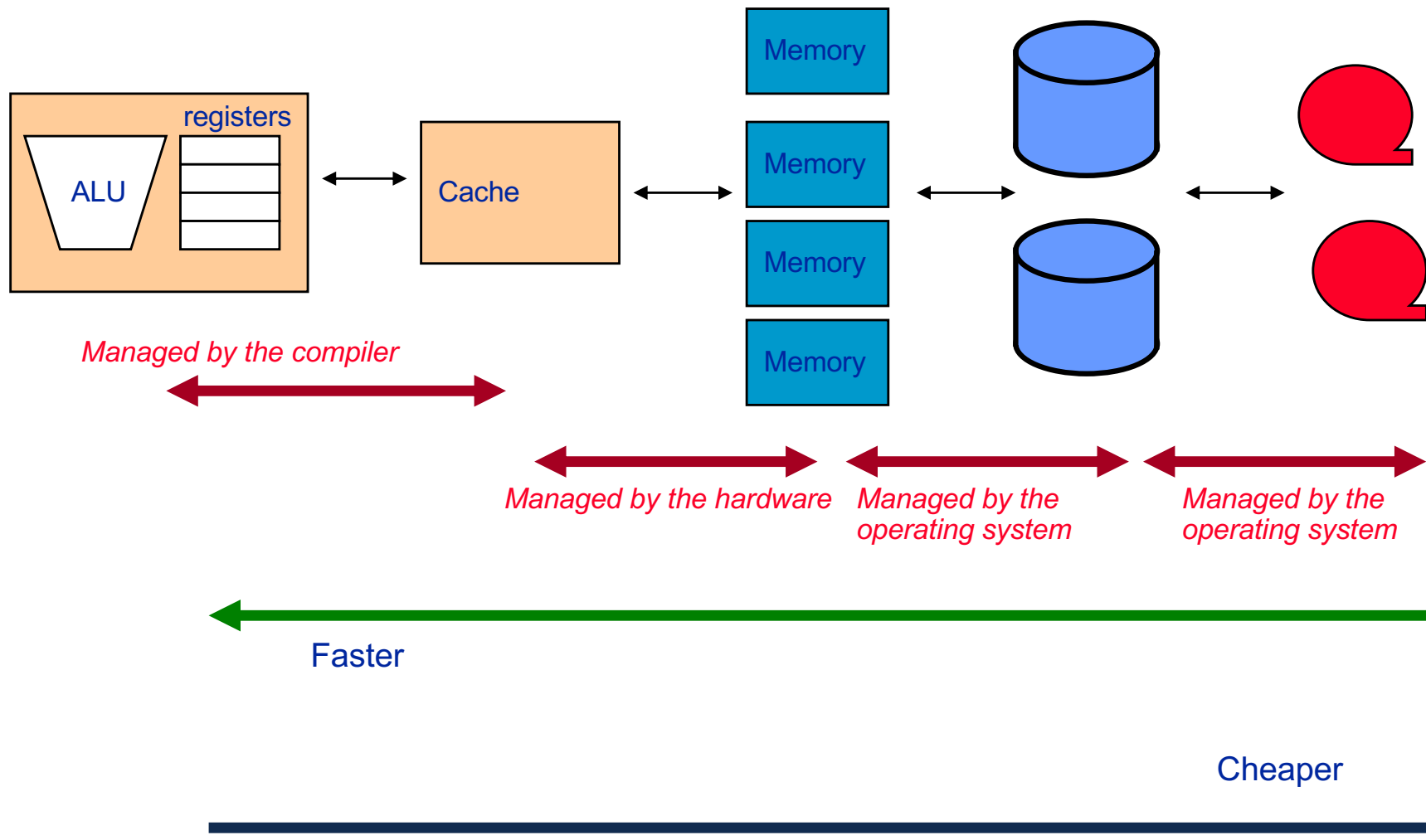




MEMORY HIERARCHY

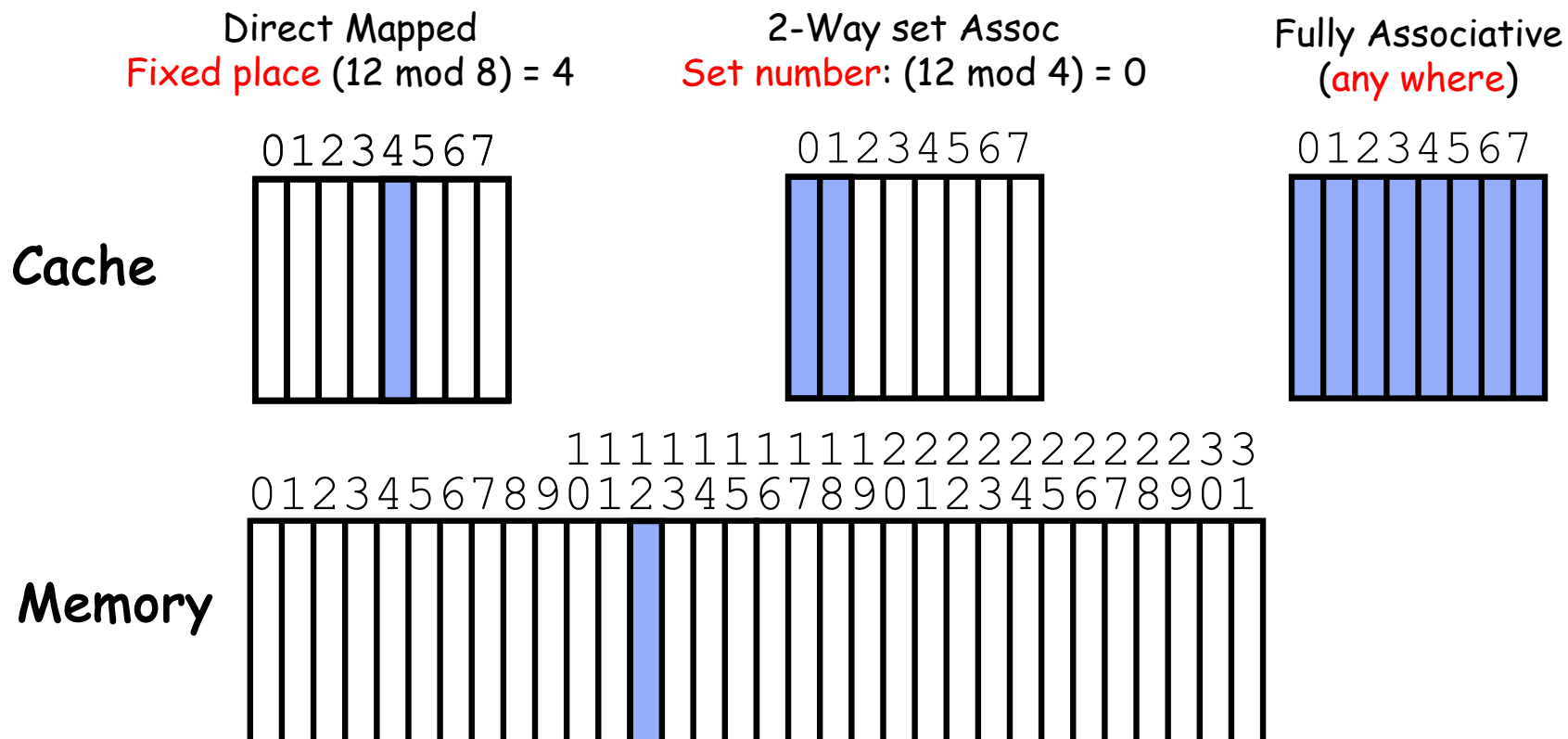
The Memory Hierarchy

Where do Solid State Disks (SSDs) fit?



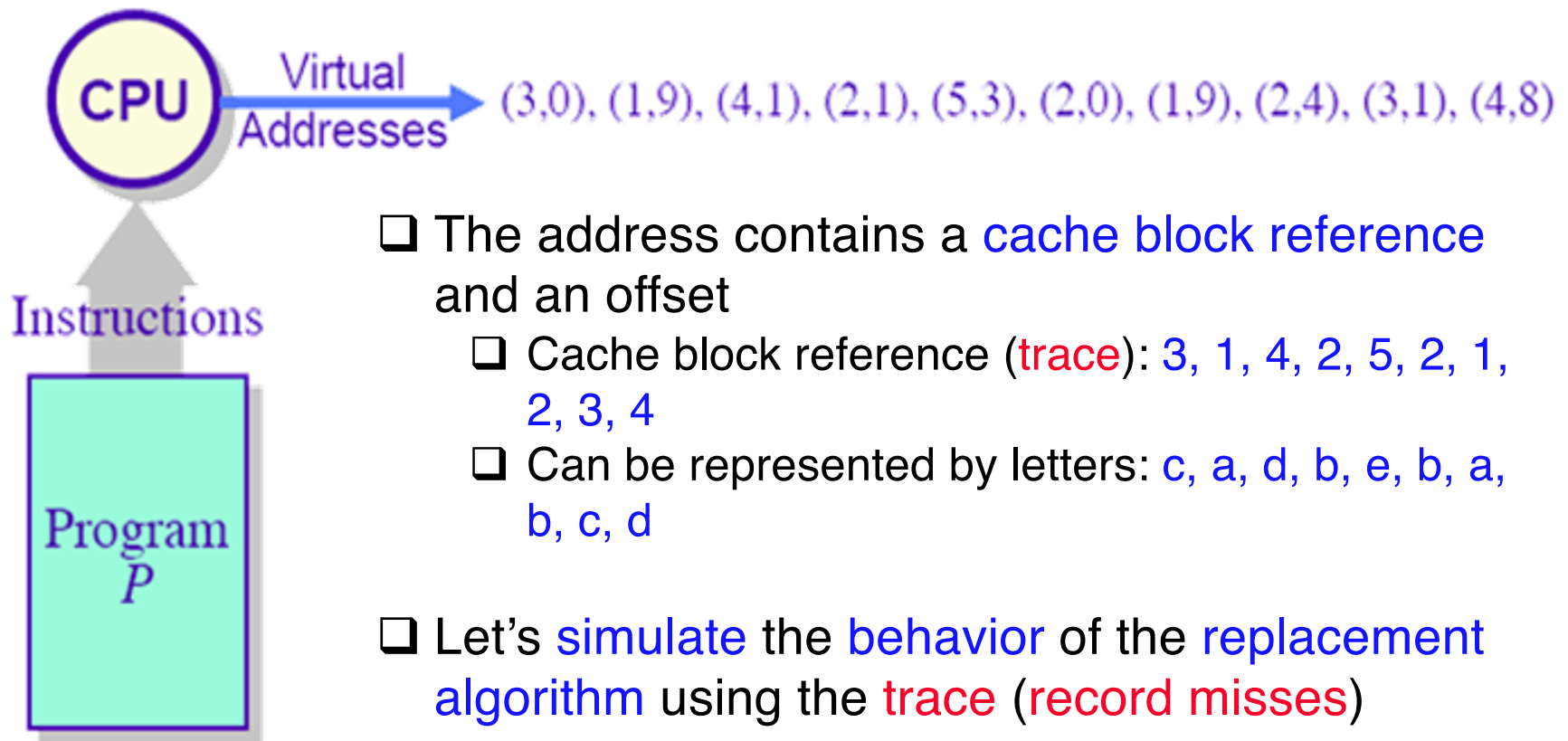
Q1: Block placement

- Block 12 placed in 8-block cache:
 - Direct mapped, fully associative, 2-way set associative



Page Replacement Algorithms

- Record a trace of the cache blocks accessed by the process



- ☐ The address contains a **cache block reference** and an offset
 - ☐ Cache block reference (**trace**): 3, 1, 4, 2, 5, 2, 1, 2, 3, 4
 - ☐ Can be represented by letters: c, a, d, b, e, b, a, b, c, d
- ☐ Let's **simulate** the **behavior** of the **replacement algorithm** using the **trace** (**record misses**)
- ☐ **Fewer** misses means **better** performance